LUCID: Statistically Grounded Self-Correction for LLM Reasoning Evaluation

Anonymous ACL submission

Abstract

Evaluating chain-of-thought reasoning at scale relies on cheap LLM raters, but their systematic rating biases propagate to downstream estimates. We introduce LUCID (**L**oop for **U**nbiased **C**alibration via **I**nference-time **D**iagnosis): we extend DSL, a doubly-robust debiasing method, to diagnose an LLM rater's structured biases from a small expert-labeled set, translate them into natural-language feedback, and let the rater re-grade itself, with no retraining, no model access, and $0.30 in API calls. On PRM800K, a single round closes GPT-5-mini's calibration gap by $\times$8.7 ($-$7.8pp to $-$0.9pp), raises error-detection $F_{1,-}$ by 32%, and improves best-of-N trajectory selection by +13.7pp, using 1,000 expert labels (SD 0.4pp across four runs). This is possible because the bias is *structured*, concentrated in computation ($-$14.6pp) and verification ($-$11.9pp) steps and in late reasoning positions ($r = -0.25$), which makes targeted textual feedback effective. DSL itself achieves 7.2$\times$ lower RMSE than expert-only at n=200 and extends to transition matrices with cluster-robust inference. The approach transfers across models (GLM-5.2: +3.1pp), datasets (ProcessBench: +5.2pp), and domains (EntailmentBank: 41% gap reduction; Mind2Web: 97%, though it over-corrects an already-calibrated rater). However, LUCID corrects *calibration* bias (systematic over- or under-rating) but not *competence* deficits: it succeeds where the rater is conservative yet capable (Mind2Web) and fails where it lacks the underlying skill (ProofWriter). Practitioners should ask whether their evaluator is biased or simply unable: LUCID fixes the former by adjusting the rater's threshold, but it cannot teach the rater to reason.

1 Introduction

Large language model (LLM) agents increasingly perform complex reasoning through chain-of-thought (CoT) traces: step-by-step explanations that precede a final answer. While these traces can improve accuracy, the reasoning itself is often *unsound*: agents may skip necessary verification steps, make logical leaps masked by fluent language, or arrive at correct answers through flawed intermediate conclusions. Evaluating the quality of agent reasoning (not just final-answer correctness) is critical for trust, but requires expensive expert annotation of individual reasoning steps. Using cheaper LLM annotators as proxies introduces systematic measurement error, a well-studied problem in computational social science (Grimmer and Stewart, 2013; Gentzkow et al., 2017), that propagates to downstream estimates if left uncorrected (Keith et al., 2020). Such LLM-generated predictions are increasingly deployed in high-stakes applications, from poverty mapping with satellite imagery and agent-retrieved text (Murugaboopathy et al., 2026; Jean et al., 2016) to automated scientific review, making reliable calibration essential (Geng et al., 2024; Shen et al., 2024; Kapoor et al.,

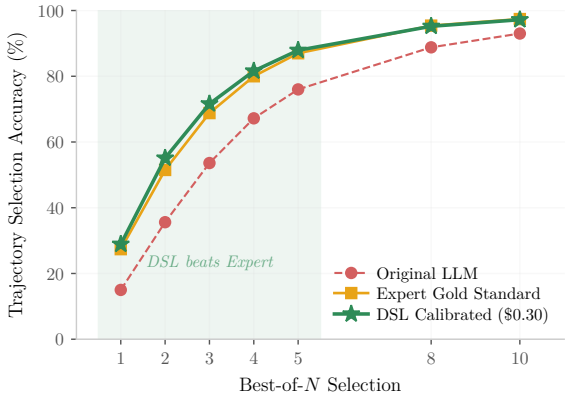


Figure 1: Downstream Best-of-N trajectory selection accuracy on 7,348 MATH problems. DSL-calibrated step scores (green) outperform expensive Expert Gold Standard labels (gold) at $N \leq 5$, where finding the correct trajectory is hardest. Gains converge at high N where most traces are already correct.

2024; Detommaso et al., 2024).

A natural approach is to use a weaker, cheaper model (e.g., GPT-5-mini) to rate step soundness at scale, then apply debiasing methods (Prediction-Powered Inference (PPI) (Kluger et al., 2025) and Design-based Supervised Learning (DSL) (Egami et al., 2023, 2024)) to correct the biased ratings using a small expert-labeled subset (see Appendix A for background). de Pieuchon et al. (2025) demonstrate this for scalar document labels; we extend it to reasoning traces. But a key question remains: *can the diagnosed bias be fed back to the agent to improve its ratings, without additional labeling?*

In this paper, we answer affirmatively. We introduce LUCID (**L**oop for **U**nbiased **C**alibration via **I**nference-time **D**iagnosis), a prompt-based self-correction framework: DSL diagnoses the agent's systematic biases (e.g., under-rating computation steps by 14.6pp), translates them into textual feedback, and the agent re-rates with this feedback. On PRM800K (Lightman et al., 2024), a single round closes the calibration gap by $\times 8.7$, improves error detection ($F_{1,-}$ +32%), and improves downstream trajectory selection by +13.7pp (Figure 1). The gain is stable across four runs (mean 85.3%, SD 0.4pp) and requires only 1,000 expert labels for diagnosis and no retraining.

Our contributions are:

1. **Prompt-based self-correction (LUCID).** A closed-loop procedure (Agent $\rightarrow$ DSL bias diagnosis $\rightarrow$ textual feedback $\rightarrow$ Agent self-correction) that reduces the calibration gap from -7.8pp to -0.9pp and improves error detection ($F_{1,-}$ +32%) through calibration rather than retraining. Unlike prior self-correction methods (Section 2), the feedback is derived from a doubly-robust statistical diagnosis rather than ad-hoc critique. We show that a single round captures the full benefit (mean 85.3%, SD 0.4pp across four runs), and that the gain is driven by the diagnosed calibration message (ablation, Section 5.1).

2. **Step-level bias diagnosis.** We apply PPI and DSL to step-level reasoning traces (594,690 steps from PRM800K with GPT-5-mini as the biased rater), extending the benchmarking of de Pieuchon et al. (2025) from document labels to reasoning steps. DSL achieves $7.2\times$ lower RMSE than Expert-only at $n = 200$ labeled steps and reaches RMSE < 0.5 with $2.4\times$ fewer labels (though this advantage is limited to $n < 1{,}000$ due to honest forest overfitting at moderate budgets; Section 5). The diagnosed bias is *structured*: concentrated in computation (-14.6pp) and verification (-11.9pp) step types and in later reasoning positions ($r = -0.25$), all reported on the held-out evaluation set. This structure is what makes textual feedback effective.

3. **DSL extensions: structured parameters and clustered inference.** We extend DSL from scalar coefficients to transition matrices over reasoning step types (with a proof of double robustness), and provide cluster-robust standard errors for within-problem correlation (3–3.5$\times$ inflation), enabling valid uncertainty quantification for large-scale agent evaluation.

Finally, we delineate the scope of prompt-based debiasing: it corrects *calibration* bias (systematic over- or under-rating) but does not fundamentally improve the rater's ability to *discriminate* correct from incorrect steps. In practice it can recalibrate a rater that is merely too conservative, but not one that lacks the skill to judge a step at all.

We validate our methods using PRM800K (Lightman et al., 2024), which contains $\sim$800K human step-level correctness labels on model-generated solutions to MATH problems (Hendrycks et al., 2021). We use GPT-5-mini (via Azure API) as the biased step rater. Additional results on iterative DSL prediction cleaning, adaptive feature selection, and retrieval-augmented generation (RAG) are reported in the appendices.

2 Related Work

Statistical debiasing of LLM annotations. de Pieuchon et al. (2025) benchmark PPI (Angelopoulos et al., 2023; Kluger et al., 2025) and DSL (Egami et al., 2023, 2024) for debiasing logistic regression coefficients from LLM annotations across 16 dataset$\times$LLM combinations, showing DSL achieves lower sRMSE at small labeled sample sizes. PPI uses gradient rectification to debias point estimates, providing valid confidence intervals regardless of prediction quality. DSL uses doubly-robust pseudo-outcomes with cross-fitted honest forests (Wager and Athey, 2017; Athey et al., 2019), connecting to the double/debiased ML literature (Chernozhukov et al., 2018; Bang and Robins, 2006; Kennedy, 2023). Recent extensions include cross-prediction-powered inference



Figure 2: LUCID overview. **(a)** GPT-5-mini systematically under-rates computation and verification steps (diagnosed by DSL on n=1,000 expert labels); the red row is a false negative that feedback corrects. **(b)** The closed loop translates the diagnosed bias profile into textual feedback; a single re-rating round closes the calibration gap from 7.8 pp to 0.9 pp (×8.7) without retraining. The dashed return arrow marks the optional additional round (one round suffices in practice), and the green check indicates the false negative in (a) flipping to correct after re-rating.

(Zrnić and Candès, 2024) and PPI with nonuniform sampling (Kluger et al., 2025). However, prior work uses document-level binary labels; we extend to step-level reasoning traces and structured parameters.

LLM self-correction and evaluation bias. Prompt-based self-correction methods (Self-Refine (Madaan et al., 2023), Reflexion (Shinn et al., 2023)) generate feedback from the model itself with no statistical grounding, targeting output quality rather than rating bias. LLM-as-a-judge evaluation is known to be systematically biased (Chang et al., 2024): judges exhibit position and verbosity biases (Zheng et al., 2023), favor their own generations (Panickssery et al., 2024), and can mimic human falsehoods (Lin et al., 2022). Existing mitigations address these at the level of prompt design or aggregation (self-consistency, Platt scaling) or direct calibration of confidence scores (Tian et al., 2023). Our loop differs: the feedback is derived from a doubly-robust statistical diagnosis with confidence intervals, targets step-level soundness *rating* bias, and predicts *which* step types will improve ($r = 0.79$).

Process supervision and agent evaluation. Lightman et al. (2024) introduce PRM800K (∼800K human step-level labels on MATH solutions), enabling process reward models. Wang et al. (2024) auto-generate step-level labels; Zheng et al. (2025) benchmark earliest-error identification. These works focus on *training* verifiers; we use these datasets to *debias* reasoning quality estimates. Agent benchmarks (AgentBoard, (Ma et al., 2024); SWE-bench, (Jimenez et al., 2024)) evaluate at the trajectory level; we propose step-level debiased evaluation with statistical guarantees.

3 Methodology

Our methodology centers on a *closed-loop self-correction procedure*: DSL diagnoses systematic bias in the agent's step ratings, translates this into textual feedback, and the agent re-rates. The supporting components are: (1) step-level soundness regression, which defines the debiasing setup; (2) transition matrix recovery via extended DSL; and (3) clustering-aware inference for valid uncertainty quantification.

3.1 LUCID: Prompt-Based Self-Correction

Procedure. Each round: (1) DSL diagnoses bias in current predictor $\hat{y}^{(r)}$ on $n_{\text{diag}} = 1{,}000$ labeled steps, (2) the bias profile (per-type biases, position correlation, overall calibration gap) is translated to textual feedback, (3) the agent re-rates $n_{\text{eval}} = 1{,}000$ steps with feedback-augmented prompts, producing $\hat{y}^{(r+1)}$, (4) stop when $\Delta^{(r)} < \epsilon = 0.001$. In practice, a single round captures the full benefit

(Section 5.1).

Feedback format. The feedback is a natural-language summary appended to the rating prompt, structured in three parts mirroring the diagnosed bias profile: *overall calibration* (*"You predict 83.8% of steps correct, but the true rate is 91.6% (you under-rate by 7.8pp overall)"*); *per-type biases* (*"You under-rate computation steps by 14.6pp and verification steps by 11.9pp"*); and *position trend* (*"You also under-rate later steps,* $r = -0.25$*"*), closing with a directive to be less conservative for those types in late positions. Providing exact percentages (rather than qualitative guidance such as "increase your ratings") lets the model apply targeted, per-type corrections rather than a uniform threshold shift; and because the profile is recomputed from the diagnostic labels each round, the feedback automatically adapts if the loop is iterated.

3.2 Step-Level Soundness Framework

We treat each reasoning step as a unit of analysis: $Y \in \{0, 1\}$ is the expert soundness label, $\hat{Y} \in \{0, 1\}$ is GPT-5-mini's rating, $X \in \mathbb{R}^p$ includes step type (6 categories), position, length, and difficulty (Section B), and θ is the logistic regression coefficient vector. This is a direct application of PPI/DSL (Appendix A): the methods take $(X, Y, \hat{Y}, \text{selected})$ and return debiased $\hat{\theta}$. Step types (Setup, Computation, Inference, Verification, Strategy, Restatement; Table 20) are classified automatically via GPT-5-mini.

3.3 Transition Matrix Recovery

DSL extends from scalar $\theta \in \mathbb{R}^p$ to structured transition matrices $\theta \in \mathbb{R}^{K \times K}$. Let $s_i \in \{1, \ldots, K\}$ denote step type. We estimate $\theta_{jk} = P(s_{i+1} = k \mid s_i = j)$ via the pseudo-outcome:

$$\tilde{y}_{i,k} = \hat{g}_k(\hat{s}_{i+1}) + \frac{b_i}{\pi}\left(y_{i,k} - \hat{g}_k(\hat{s}_{i+1})\right), \qquad (1)$$

where $y_{i,k} = \mathbf{1}\{s_{i+1} = k\}$ and $\hat{g}_k$ is a cross-fitted classifier predicting $P(s_{i+1} = k \mid \hat{s}_{i+1})$. The estimator $\hat{\theta}_{jk} = \sum_{i:s_i=j} \tilde{y}_{i,k} / \sum_{i:s_i=j} 1$ is doubly robust (Proposition in Appendix C). Rows are normalized to sum to 1; this preserves consistency (and hence double robustness) by the continuous mapping theorem, since each entry is consistent and each true row sums to 1.

3.4 Clustering-Aware Inference

Steps within the same trace are correlated: if step i is wrong, step $i + 1$ is likely wrong too. Point estimates remain consistent, but naive standard errors underestimate uncertainty. We use cluster-robust sandwich estimators (Liang and Zeger, 1986) with problem-level clusters:

$$\hat{V} = \hat{A}^{-1}\hat{B}\hat{A}^{-1}, \quad \hat{B} = \sum_{g=1}^{G} S_g S_g^\top, \qquad (2)$$

where $S_g = \sum_{i \in g} m(\tilde{y}_i, x_i; \hat{\theta})$ is the cluster-level score. Naive SEs are 3–3.5$\times$ too small at $n = 1{,}000$ (Table 25).

4 Experiments

Data. We use PRM800K (Lightman et al., 2024): $\sim$800K human step-level correctness labels on MATH (Hendrycks et al., 2021) solutions. After binarizing ($Y = 1$ if rating $= +1$) and filtering, we obtain $N = 594{,}690$ steps across 90,340 traces and 11,267 problems. We use GPT-5-mini (OpenAI, 2025) (Azure API, temperature 0) as the biased step rater for all N steps. ProcessBench (Zheng et al., 2025) (3,400 traces, 16,548 steps) serves as an out-of-sample evaluation set.

Features. Each step has $p = 9$ features: step type (5 one-hot dummies), normalized position, log step length, and difficulty (2 dummies; see Section B). An augmented set ($p = 24$) adds embedding cluster features.

Design. *Vary-expert*: fix $N = 594{,}690$, vary n from 200 to N (10 log-spaced points, 30 repetitions). *Self-correction*: $n_{\text{diag}} = 1{,}000$ labeled steps for diagnosis, $n_{\text{eval}} = 1{,}000$ disjoint steps for evaluation. *Transition matrix*: estimated at $n = 1{,}000$ on consecutive step pairs. Methods compared: Reference (all-N expert), Expert-only (n expert), Imputation (all-N predicted), PPI, and DSL (Table 31, Appendix). Metrics: sRMSE, relative Frobenius norm, CI coverage, and cluster-robust vs. naive SEs.

5 Results

We present results using GPT-5-mini as the biased step rater ($\hat{Y}$). Self-correction experiments use $n_{\text{diag}} = 1{,}000$ labeled steps for bias diagnosis and $n_{\text{eval}} = 1{,}000$ disjoint steps for evaluation. DSL experiments use 30 random repetitions; error bars

show ± 1.96 SE. Table 1 summarizes the headline results.

Setting	Original	Corrected	Δ	Cost
PRM800K (GPT-5-mini)	79.0%	85.7%	+6.7pp	$0.30
PRM800K (GLM-5.2)	84.4%	87.5%	+3.1pp	$0.30
ProcessBench	81.4%	86.6%	+5.2pp	$0.30
DSL RMSE (n=200)	6.09	0.84	7.2×	—

Table 1: Headline results. Self-correction improves step-rating accuracy across models and datasets at negligible API cost. DSL provides 7.2× lower RMSE than Expert-only at $n = 200$ labeled steps.

5.1 LUCID: Prompt-Based Self-Correction

A single round of LUCID closes the calibration gap from −7.8pp to −0.9pp (×8.7 reduction), improves $F_{1,-}$ on unsound steps by +32% (0.146→0.192), and the LUCID-corrected ratings reduce the Brier score from 0.210 to 0.143 (−32%); Table 28 (Appendix) summarizes these metrics across settings. The gain is stable across four independent runs (mean 85.3%, SD 0.4pp; Table 23). A qualitative example showing feedback flipping a false negative is in Table 29 (Appendix). Per-type calibration gaps close substantially: computation (14.6pp→3.6pp), verification (11.9pp→3.0pp), inference (10.8pp→6.2pp); see Figure 3 and Table 2. To guard against the small evaluation set, we replicate single-round self-correction on an independent 5,000-step evaluation set, disjoint from both the original evaluation and labeled sets. The improvement is +6.2pp (95% CI [4.7, 7.8]), consistent with the +6.7pp headline; per-type gains remain largest for verification (+14.3pp) and computation (+10.0pp), and the per-type bias–gain correlation retains its sign ($r = 0.54$ across six types). The loop is also cheap: it adds only 1,000 API calls ($0.30) on top of the 1,000 labels ($500) required for diagnosis, comparable to expert-only at n=1,000 but reaching 85.7% vs. 80.1%, whereas full expert annotation costs $300,000 and fine-tuning a verifier costs $50+GPU but requires retraining for new bias patterns (Table 21, Appendix).

Per-type and position improvements. The LUCID-corrected ratings most effectively improve the step types with the largest under-rating: computation (bias −14.6pp, gain +10.4pp) and verification (bias −11.9pp, gain +9.0pp). Types already well-calibrated (setup, strategy) see minimal improvement (+5.0pp and +2.6pp; Table 26). GPT-5-mini

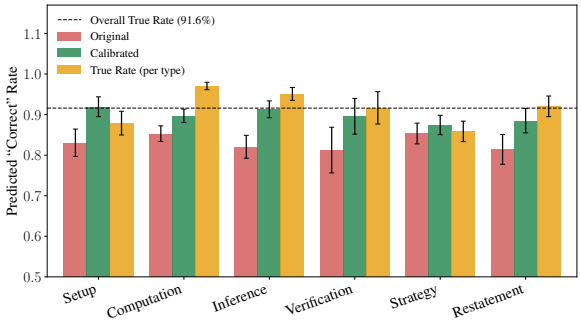


Figure 3: Calibration by step type (with 95% bootstrap CIs). Predicted vs. true "correct" rates before and after one round of feedback. The dashed line marks the overall true rate (91.6%).

Metric	Original	Corrected
Brier score ↓	0.210	0.143
Calibration gap (overall) ↓	7.8pp	0.9pp
Computation gap ↓	14.6pp	3.6pp
Verification gap ↓	11.9pp	3.0pp
Inference gap ↓	10.8pp	6.2pp

Table 2: Calibration metrics. Self-correction reduces the calibration gap by ×8.7 overall and by up to ×4.1 for the most biased step types.

systematically under-rates later steps (position 0.8–1.0: 60.0% pred rate vs. 93.3% true rate); after feedback, this quintile improves by +25.6pp. The step type × position interaction (Figure 9, Appendix) shows that computation and verification steps in late positions are most biased.

Analysis. Per-type improvement correlates with per-type bias ($r = 0.79$, $p = 0.06$). The association is directionally consistent with targeted recalibration rather than uniform threshold-shifting, but with only six step types it is not statistically significant ($p = 0.06$), so we interpret this cautiously. An independent 5,000-step replication shows the same sign ($r = 0.54$), confirming the direction but not the magnitude. Computation bias worsens monotonically with position, from 0pp (early) to −9.1pp, −11.6pp, −15.8pp, and −22.6pp (late), so position and type biases interact. Correctability is highest for the most biased steps: 65.2% of errors at position 0.8–1.0 are fixed by feedback, versus 30.0% at position 0–0.2. Computation steps are most correctable (59.3%); strategy (39.4%) and restatement (37.5%) are least.

Downstream validation: trajectory filtering. We evaluate best-of-N trajectory filtering: for each problem with multiple traces, we rank by average

step-level score and select the top-N. Table 3 shows that DSL-calibrated labels improve top-1 selection accuracy by +13.7pp over original GPT-5-mini labels (15.2%→28.9%) and match expert labels (27.5%). The gap widens for best-of-2 (35.6%→55.1%, +19.5pp). Full results across all N values in Table 22 (Appendix). Note that trajectory filtering requires calibrated probabilities for all 594,690 steps, which the prompt loop (limited to 1,000 binary eval labels) cannot provide; we therefore use the DSL honest forest, which produces calibrated probabilities via cross-fitting. Both the prompt loop and the honest forest share the same DSL bias diagnosis: the loop translates it into textual feedback, while the forest uses it to correct predictions directly. Debiased step scores also transfer downstream: we train process reward models (PRMs) on three label sources (original GPT-5-mini, DSL-debiased, and expert/oracle). All PRMs collapse to majority-class prediction (91.6% accuracy, $F_{1,-} = 0$) under the limited feature set, yet their *calibration* differs: debiased training reduces ECE by 56% (0.084→0.037), moving toward the oracle (0.015), and for trajectory selection debiased-trained PRMs improve top-1 accuracy by +1.8pp over original-trained PRMs (17.0%→18.8%), confirming that debiased labels yield better-calibrated scoring functions for ranking. Since expert-trained PRMs calibrate best (ECE 0.015), the debiased-label PRM's approach to oracle performance is a label-efficient substitute for expert annotation, where expert step labels are the dominant cost.

Scoring Method	Top-1	Top-2	Top-4
Expert (gold standard)	27.5%	51.6%	80.0%
GPT-5-mini (original)	15.2%	35.6%	67.1%
DSL calibrated	28.9%	55.1%	81.6%

Table 3: Best-of-N trajectory selection accuracy on 7,348 problems. DSL-calibrated step scores improve top-1 selection by +13.7pp over original GPT-5-mini scores.

Why DSL can match the expert oracle. Two caveats explain why a calibrated version of a biased rater can match the expert curve. First, selection is scored against final-answer correctness, not step soundness: a trace with some flawed steps can still reach the right answer, so averaging binary step labels is only a coarse proxy for trajectory quality. Second, the expert curve is an oracle that uses all ground-truth step labels (unavailable in practice); the DSL honest forest approximates it from 1,000 diagnostic labels, and its calibrated probabilities provide a finer ranking signal than averaging binary labels.

Baselines. Because the evaluation set has a 91.6% base rate of correct steps, accuracy-maximizing baselines (Platt scaling, fine-tuned LR/MiniLM) reach 91.6% by detecting *no* errors ($F_{1,-} = 0.00$). Self-correction instead improves error detection: it halves false alarms (144→76) while maintaining recall, raising $F_{1,-}$ from 0.146 to 0.192 (+32%). A global threshold shift baseline (matching the true positive rate) reaches 84.6% accuracy but $F_{1,-}$ drops to 0.083. Per-type feedback adds substantial value beyond global recalibration. Fine-tuned Qwen3.5-LoRA (Qwen Team, 2026; Hu et al., 2021) verifiers (0.8B–9B) detect only a fraction of unsound steps ($F_{1,-}$ 0.18–0.24) despite detection-oriented training (Table 24, Appendix). Self-correction is therefore preferable to fine-tuned verifiers when calibrated probability estimates (not binary labels) are needed for downstream ranking, where overconfidence is harmful; when the biased predictor is a closed-source API model with no fine-tuning access; when rapid adaptation to new bias patterns is needed, since self-correction requires only a new diagnostic round ($0.30) rather than full retraining; or when the same feedback must transfer across model families without retraining (GLM-5.2: +3.1pp, Section 5.2). On a class-balanced subset (Appendix J) accuracy improves only 57.1%→58.3% ($F_{1,-}$ 0.333→0.327) — a marginal gain within run-to-run noise — so the headline accuracy gain is largely a class-imbalance artifact and the real benefit is calibration. For pure accuracy on balanced datasets with fine-tuning access available, a fine-tuned classifier is preferable, but this is not our setting. The deeper distinction is that self-correction improves *calibration* (matching predicted rates to true rates), which is what downstream ranking tasks require, while accuracy-maximizing baselines achieve their scores by collapsing to the majority class and detecting no errors, a form of discrimination failure that calibration directly addresses.

Ablation. Re-rating with partial feedback: overall calibration alone (+6.9pp), type biases alone (+6.2pp), position alone (+1.9pp), full feedback (+5.3pp). The gaps *among* variants (0.6–1.6pp) are within run-to-run variance (SD 0.4pp), so the

specific feedback format matters little. However, the overall, type-only, and full-feedback variants each beat the no-feedback baseline (79.0%) by +5–7pp, more than 10× the SD; even position-only (+1.9pp), which isolates a comparatively weak bias dimension, clears the SD. It is this baseline gap, not the within-variant differences, that shows the diagnosis drives the loop (Table 27, Appendix).

5.2 Generalization

All results above use GPT-5-mini (OpenAI, 2025) on PRM800K (Lightman et al., 2024). We next test whether the same DSL-diagnosed feedback transfers to other models, datasets, and domains.

Cross-model transfer. The same DSL-diagnosed feedback (generated from GPT-5-mini's bias profile) improves GLM-5.2 (ZAI, 2026) by +3.1pp (84.4%→87.5%), confirming model-agnostic transfer. GLM-5.2 is better calibrated overall (−1.4pp vs. −7.8pp) but still under-rates computation and verification steps (Table 32, Appendix).

Cross-dataset transfer (ProcessBench). On ProcessBench (16,548 steps across GSM8K, MATH, OlympiadBench, OmniMath), self-correction improves accuracy from 81.4% to 86.6% (+5.2pp). Gains scale with initial bias severity: OmniMath (worst-calibrated, 72.3%) improves most (+10.7pp), while GSM8K (near ceiling, 94.6%) slightly degrades (−2.2pp). Self-correction outperforms few-shot baselines (k=1–10) by up to +3.9pp (Table 34, Appendix).

Multi-domain. To test whether the framework generalizes beyond mathematical reasoning, we apply the same loop to three non-mathematical domains: for each, we define a domain-specific step taxonomy, rate steps with GPT-5-mini, diagnose bias on n=500 labeled steps, and re-rate n=500 eval steps with feedback. Table 4 summarizes results. The most striking result is Mind2Web: GPT-5-mini rates only 10.8% of correct agent actions as correct (calibration gap = 89.2pp); after feedback, accuracy jumps to 97.4% (gap collapses to 2.6pp, 97% reduction). This suggests LLMs are profoundly uncertain when evaluating agent actions (clicking, typing, navigating) relative to mathematical reasoning, but that this uncertainty is *calibratable*: once told the true rate is near 100%, the model sharply lowers its threshold. This shows that extreme conservatism is a threshold effect, not a competence deficit. The boundary is sharpened by the balanced Mind2Web run (Table 4): once the model is already calibrated, the same feedback over-corrects rather than improving discrimination; the loop is most valuable precisely when a large, one-directional calibration gap exists to close. On EntailmentBank, the framework transfers with +2.4pp improvement and 41% gap reduction. On ProofWriter, self-correction provides no improvement (−0.4pp); there the errors reflect a lack of reasoning skill rather than a miscalibrated threshold, which feedback cannot supply. The diagnosed bias profiles make this concrete (Table 18; these are diagnostic-set biases, while Table 4 reports the eval-set accuracy gaps): Mind2Web exhibits a large, uniform conservatism (overall −84pp; action execution −83pp, termination −98pp) that a single threshold correction fixes, whereas ProofWriter has a small overall diagnosed gap (−4pp) but a localized derivation bias (−33pp) that feedback cannot fix, a competence deficit rather than a miscalibrated threshold.

Figure 4 visualizes how LUCID pulls the severest per-type biases toward zero.

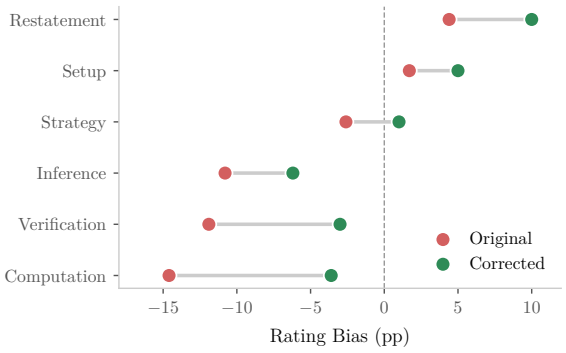


Figure 4: Prompt-based self-correction reduces structured rating biases on PRM800K. Red dots show GPT-5-mini's original under-rating; green dots show calibrated ratings after feedback. The severest biases (computation −14.6pp, verification −11.9pp) are pulled toward zero.

5.3 DSL as Bias Diagnostic

Self-correction is only as good as the diagnosis it consumes, so we examine DSL as a bias diagnostic in its own right.

Vary-expert. DSL achieves 7.2× lower RMSE than Expert-only at $n = 200$ (0.84 ± 0.08 vs. 6.09 ± 0.99; paired t-test, $p < 0.001$) and reaches RMSE < 0.5 at $n = 486$ (2.4× fewer labels than Expert-only). At $n \geq 2{,}875$, Expert-only significantly outperforms DSL ($p < 0.001$), the crossing

Domain	Orig Acc	Corr Acc	Δ	Gap Orig	Gap Corr
PRM800K (math) (Lightman et al., 2024)	79.0%	85.7%	+6.7pp	7.8pp	0.9pp
EntailmentBank (science) (Dalvi et al., 2021)	94.2%	96.6%	+2.4pp	5.8pp	3.4pp
Mind2Web (agents, calibration) (Deng et al., 2023)	10.8%	97.4%	+86.6pp	89.2pp	2.6pp
Mind2Web (agents, balanced) (Deng et al., 2023)	50.8%	61.0%	+10.2pp	0.4pp	38.6pp
ProofWriter (logic) (Tafjord et al., 2021)	94.4%	94.0%	−0.4pp	4.8pp	4.8pp

Table 4: Multi-domain generalization. The framework transfers strongly to calibration but over-corrects on already-calibrated data and fails on reasoning-competence deficits.

phenomenon, in which DSL's honest forest overfits at moderate labeling budgets (Figure 5, Table 30, Appendix).

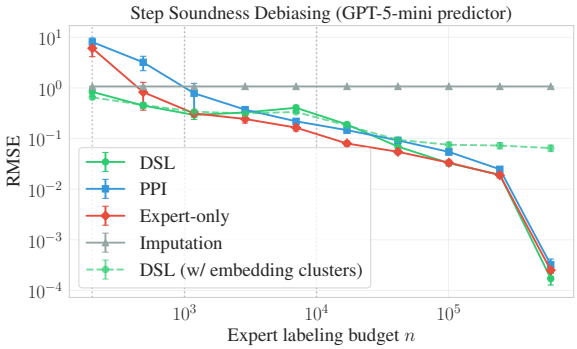


Figure 5: RMSE vs. expert sample size n for step-level soundness debiasing (GPT-5-mini predictor). Error bars show 95% CIs over 30 repetitions. DSL dominates at small n and converges to near-zero RMSE.

Beyond scalar coefficients, DSL recovers the 6×6 step-type transition matrix at $n = 1{,}000$ with Relative Frobenius norm 0.031 (95% CI $[-0.004, 0.065]$); the biased matrix (from GPT-5-mini predictions) is nearly uniform, erasing all structure, and DSL restores it (Figure 12, Appendix). Cluster-robust standard errors are likewise 3–3.5× larger than naive SEs at $n = 1{,}000$ (Table 25), reflecting substantial within-problem correlation that invalidates iid-based confidence intervals. The diagnosis also tolerates realistic noise: even with 30% random perturbation of step types (50 trials) the bias direction is preserved for all six types (computation remains -13.3 ± 1.2pp vs. -14.6pp unperturbed; verification -10.1 ± 3.3pp vs. -11.9pp), and injecting 10–30% random noise into predictions leaves accuracy flat (80.4% baseline, 78.6–81.2% corrected; at most +0.8pp at 20% noise, vs. +6.7pp with structured bias), so the method does not rely on spurious patterns. The limits are instructive: of 84 unsound steps, 58 remain undetected after correction (recall$_-$ = 0.202), concentrating in computation (18) and strategy (15) steps, precisely the types where reasoning errors are most subtle (e.g., a correct-looking algebraic manipulation that silently assumes an unjustified constraint). The feedback corrects *calibration* but cannot teach the model to recognize deeply flawed reasoning superficially resembling correct work: it addresses systematic *rating* bias, not *reasoning* deficiency.

6 Conclusion

We have presented LUCID, a prompt-based self-correction framework for debiasing LLM reasoning evaluation: DSL diagnoses systematic biases, translates them into textual feedback, and the agent re-rates. No retraining, no fine-tuning, no additional expert labels beyond the small diagnostic subset.

The method improves GPT-5-mini's step-rating accuracy from 79.0% to 85.7% (+6.7pp, stable across four runs), narrows the calibration gap nearly ninefold, and transfers across model families (GLM-5.2: +3.1pp) and datasets (ProcessBench: +5.2pp). DSL serves as an effective bias diagnostic: 7.2× lower RMSE than Expert-only at $n = 200$, with the diagnosed bias structured by step type and position. Extended DSL recovers transition matrices (Frobenius error 0.031) and cluster-robust inference inflates SEs by 3–3.5×, enabling valid uncertainty quantification.

Prompt-based self-correction addresses *calibration* bias (systematic over- or under-rating) but does not fundamentally improve the model's ability to *discriminate* correct from incorrect steps: it can recalibrate a rater that is merely too conservative, but it cannot supply reasoning skill the rater lacks.

Future work. Extending to additional model families, per-problem adaptive feedback, combining self-correction with fine-tuned verifiers, and formal validation of the step type taxonomy.

7 Limitations

Calibration vs. discrimination. Our framework addresses calibration bias (systematic over- or under-rating) but does not fundamentally improve the model's ability to *discriminate* correct from incorrect steps. The second Mind2Web run (balanced data with negative examples) demonstrates this: when the model is already well-calibrated, feedback improves accuracy (+10.2pp) but overcorrects on calibration (gap +38.6pp) and destroys error detection ($F_{1,-}$ drops to 0.000). The framework is therefore most effective when there is a genuine calibration bias to correct (e.g., Mind2Web gold trajectories with 89.2pp gap), and less effective when the model lacks the underlying reasoning ability to evaluate steps (e.g., ProofWriter). Practitioners should assess whether their evaluator's errors stem from conservatism (fixable by our method) or from reasoning deficits (requiring different interventions).

Baseline comparison. Self-correction (+6.7pp) is outperformed in raw accuracy by simple calibration baselines (Platt scaling: +12.6pp) because the evaluation set is heavily imbalanced (91.6% correct steps). A trivial "always predict 1" baseline achieves 91.6%, far above self-correction's 85.7%. These baselines win on accuracy by detecting no errors ($F_{1,-} = 0.00$); self-correction instead improves error detection ($F_{1,-}$ from 0.146 to 0.192), which is valuable for downstream tasks (process reward models) but may not be optimal when pure accuracy is the goal.

Baselines and features. Our experiments span two model families: GPT-5-mini as the biased step rater and GLM-5.2 as a transfer test (Section 5.2), where the same DSL-diagnosed feedback transfers (+3.1pp). Two families is limited coverage, however; models such as Claude or DeepSeek may exhibit different systematic biases, and the effectiveness of prompt-based self-correction may vary, so evaluating additional families is left to future work. We fine-tune two verifiers on the same 1,000 labeled steps: logistic regression on the baseline feature set ($n = 200, 500, 1{,}000$) and a MiniLM sentence-transformer classifier ($n = 1{,}000$). Both reach 91.6% accuracy, the majority-class ceiling, meaning they predict "correct" for every step (Section J). The baseline features do not separate sound from unsound steps, so fine-tuned classifiers cannot beat a trivial baseline; the gap between self-correction (85.7%) and the ceiling therefore reflects the information content of the features rather than a limitation of prompting. We also fine-tune four open-weight LLMs (Qwen3.5-0.8B/2B/4B/9B) on the same 1,000 labeled steps with LoRA and a class-weighted loss, selecting the checkpoint by $F_{1,-}$ on a held-out validation set rather than by accuracy (Section J). Even under detection-oriented training, they detect only a fraction of the unsound steps ($F_{1,-}$ 0.18–0.24): the best (0.8B) reaches $F_{1,-}$ 0.238, marginally above self-correction's 0.192, but at substantially lower accuracy (81.4% vs. 85.7%) and, critically, produces only binary labels with no calibrated confidence scores. This makes them unsuitable for downstream ranking tasks (trajectory filtering, process reward models) where calibrated probabilities matter. Beyond the GPU infrastructure required, fine-tuned verifiers must be retrained for each new bias pattern or model family, whereas prompt-based self-correction adapts instantly via a new diagnostic round. Our approach is most advantageous when: (a) calibrated probability estimates (not binary labels) are needed for downstream ranking, (b) the biased predictor is a closed-source API model with no fine-tuning access, (c) rapid adaptation to new bias patterns is required without retraining, or (d) the same feedback must transfer across model families (GLM-5.2: +3.1pp, Section 5.2) without retraining. We treat PRM800K's expert labels as ground truth and do not empirically test the prospective mitigations (multiple annotator pools, annotation-noise models) discussed in the Ethical Considerations; doing so requires additional annotation and is left to future work. Separately, the "hard" difficulty dummy is constant zero in all PRM800K experiments because the train split lacks difficulty metadata, so difficulty effects are not identified; we retain the feature only for interface consistency.

Self-correction ceiling. Even after a single round of feedback, accuracy plateaus at 85.7%, leaving a 5.9pp gap from the true 91.6% rate. Residual errors are concentrated in computation steps (34 false negatives) and inference steps (26 false negatives), where the bias is most severe and the feedback may be insufficiently specific. Closing this gap likely requires per-step-type feedback with finer granularity or combining prompt-based correction with statistical post-processing.

Run-to-run variance. We ran four independent single-round runs with identical steps, prompts, and feedback: corrected accuracy was 85.7%, 85.5%, 85.1%, and 84.9% (mean 85.3%, SD 0.4pp; improvements +5.9 to +6.7pp). All other reported accuracies are from a single run per condition. The qualitative conclusions, that self-correction improves accuracy by roughly +6pp, that baselines predicting the majority class reach the 91.6% ceiling, and that improvement correlates with diagnosed bias, are stable across runs; point estimates should be interpreted with this variance in mind.

Taxonomy. The 6-type step taxonomy is designed by the authors and applied via GPT-5-mini classification, but we do not report inter-annotator agreement with human annotators. The taxonomy may not capture all relevant reasoning distinctions, and different granularity (e.g., splitting computation into algebraic vs. arithmetic) could change the bias profile. To assess potential misclassification, we examined edge cases in GPT-5-mini's step type assignments: 20% of computation steps are shorter than 50 characters (e.g., "That's our answer"), where the model may conflate brief computation with restatement or strategy, and step types show substantial overlap in length (mean 83–109 chars), so length alone cannot distinguish types. Yet the diagnosed bias patterns match linguistic expectations: computation and verification steps are most under-rated (both require rigorous logical checking that the model penalizes), while setup and strategy are well-calibrated (both involve pattern recognition that the model handles well). This consistency indicates the taxonomy captures meaningful distinctions despite potential individual mitigations. We note that the DSL framework is not limited to discrete step types: it accepts any feature set, including continuous features (e.g., step length, embedding clusters) or finer-grained type taxonomies. Our current 6-type taxonomy is a practical choice that captures the dominant bias dimensions; extending it is straightforward within the existing framework.

Bias profile stability. The bias profile is computed once on the full dataset and applied uniformly. If the model's bias changes across problem types, difficulty levels, or prompt formulations, the feedback may become stale. Per-batch or per-problem adaptive feedback could address this but requires more labeled data per batch. Extending PPI/DSL to handle nonuniform sampling (Kluger et al., 2025) or leveraging multiple prediction models (Zrnić and Candès, 2024) are promising directions for adaptive debiasing.

Overfitting at moderate n. DSL's cross-fitting model overfits at moderate labeling budgets ($n \geq 2{,}875$), where Expert-only significantly exceeds DSL in RMSE (paired t-test, $p < 0.001$; Table 30). This limits DSL's advantage to low-budget settings ($n < 1{,}000$). We note that our step-type features are one-hot encoded (orthogonal by construction), which reduces multicollinearity risk compared to continuous feature sets; the overfitting is driven by the cross-fitting model's capacity rather than feature correlation. Hyperparameter tuning (e.g., reducing tree depth, increasing minimum leaf size) would likely mitigate this effect.

Failure modes. Self-correction may fail or degrade when: (a) the biased predictor's errors are unstructured (random noise rather than systematic bias), since DSL cannot diagnose what has no pattern; (b) the model cannot follow feedback instructions (e.g., smaller models may ignore complex directives); (c) the bias profile shifts across domains or prompt formulations, making the diagnosed feedback stale; (d) the diagnosis set is too small ($n_{\text{diag}} < 500$), yielding noisy bias estimates that lead to inaccurate feedback. We observe (d) empirically on the multi-round pipeline: at $n_{\text{diag}} = 500$, improvement drops to +4.4pp from +5.3pp at $n_{\text{diag}} = 1{,}000$.

8 Ethical Considerations

Our framework makes LLM evaluation cheaper and more accessible, but like any debiasing tool, it carries risks that practitioners should understand.

Positive societal impacts. By reducing the cost of reliable LLM evaluation from hundreds of expert labels to just 1,000 ($0.30 in API calls), our method democratizes access to calibrated evaluation. Research groups and practitioners who cannot afford large-scale expert annotation can now calibrate their LLM evaluators at inference time. The closed-loop design (no retraining, no model access) makes it applicable to closed-source API models, broadening its reach beyond organizations with GPU infrastructure. This could accelerate the development of reliable AI systems in resource-constrained settings, including education, healthcare, and scientific research.

Risks of misuse and false objectivity. Debiasing techniques can create a veneer of objectivity: a model whose ratings have been "corrected" may appear more trustworthy than it deserves, potentially legitimizing flawed reasoning evaluations. If the bias diagnosis is inaccurate (e.g., due to a too-small labeled set or distribution shift), the feedback may introduce *new* biases rather than removing existing ones. LLM evaluation is known to be sensitive to prompt formulation and model version (Chang et al., 2024), and debiased ratings should be treated as calibrated estimates with uncertainty, not as ground truth.

Reward hacking and deployment risks. A subtler risk arises when debiased evaluations are used in high-stakes deployment pipelines (e.g., selecting reasoning traces for reinforcement learning or filtering agent outputs for production). If practitioners treat debiased ratings as equivalent to expert judgment, they may deploy models whose residual errors were masked by an imperfect correction, a form of "reward hacking" where the evaluation metric is optimized but the underlying capability is not. We mitigate this by reporting uncertainty (confidence intervals on bias estimates, cluster-robust standard errors) and by validating downstream utility (trajectory filtering, Table 3), but practitioners should maintain human oversight for safety-critical applications and periodically re-validate the bias diagnosis as models are updated.

Expert ground truth and annotation bias. Our framework treats the expert labels (PRM800K human annotations) as ground truth, but expert annotations are themselves subject to measurement error and inter-annotator disagreement. If the expert labels contain systematic biases (e.g., annotators consistently over-rate certain step types or are influenced by problem formatting), the bias diagnosis will be computed relative to a biased reference, potentially "correcting" the LLM toward a different bias rather than toward objective soundness. The framework can be adapted to this setting by: (1) using multiple annotator pools and measuring inter-annotator agreement as a diagnostic; (2) treating the expert labels as a *noisy* reference and incorporating annotation noise models into the DSL pseudo-outcome; and (3) periodically auditing the expert labels for demographic or formatting biases that could propagate through the debiasing pipeline.

Fairness and representativeness. The step-type taxonomy and bias diagnosis are computed from the labeled subset, which may not represent all problem types or difficulty levels equally. If certain problem categories are underrepresented in the labeled set, the feedback may not correct biases affecting those categories. The framework does not currently account for protected attributes in problem content (e.g., cultural context in word problems), which could interact with the bias diagnosis in ways that disproportionately affect certain user groups.

Documentation of artifacts. We provide documentation for the main artifacts used in this study, including the datasets and model families evaluated in the main experiments and appendices. The evaluation spans mathematical reasoning, scientific entailment, and web-agent tasks, with additional checks on balanced and calibration-heavy settings; however, the study is not intended as a survey of demographic representation or linguistic diversity, and we do not make claims about demographic coverage beyond what is already described in the underlying benchmark documentation. For reproducibility, we report the exact model versions, prompting setup, feature specifications, and experimental parameters in the appendices, but the paper does not include separate artifact documentation for demographic groups or multilingual phenomena because those dimensions are not central to the tasks studied.

Environmental impact. Self-correction requires one additional inference pass over the evaluation set (1,000 API calls per round), which is negligible compared to the original rating pass. This is substantially more efficient than fine-tuning a separate verifier, which requires GPU training and repeated retraining for new bias patterns. The net environmental impact is positive: fewer expert annotations and no GPU training.

Limitations of the framework. The method assumes bias is *structured* and *diagnosable* from a small labeled set. If a model's errors are unstructured (random noise) or if the bias shifts rapidly across domains, the feedback may be ineffective or stale. The framework is designed for step-level soundness evaluation in mathematical reasoning; generalization to other domains (code, planning, open-ended generation) requires domain-specific taxonomies and validation. Our multi-domain

study (Section 5.2) shows the framework transfers well to agent evaluation and scientific entailment but fails on formal logical deduction, delineating its applicability boundary.

Use of AI assistants. AI assistants were used for coding, data analysis, and manuscript-editing assistance. The language models evaluated in this study (e.g., GPT-5-mini, GLM-5.2) are research objects and are therefore outside the scope of this disclosure. The authors reviewed all AI-assisted output and take full responsibility for the content of this paper.

References

Anastasios N. Angelopoulos, Stephen Bates, Clara Fannjiang, Michael I. Jordan, and Tijana Zrnic. 2023. Prediction-powered inference. *Science*, 382(6671):669–674.

Susan Athey, Julie Tibshirani, and Stefan Wager. 2019. Generalized random forests. *Annals of Statistics*, 47:1179–1203.

Heejung Bang and James Robins. 2006. Doubly robust estimation in missing data and causal inference models. *Biometrics*, 61:962–73.

Yupeng Chang, Xu Wang, Jindong Wang, Yuan Wu, Linyi Yang, Kaijie Zhu, Hao Chen, Xiaoyuan Yi, Cunxiang Wang, Yidong Wang, Wei Ye, Yue Zhang, Yi Chang, Philip S. Yu, Qiang Yang, and Xing Xie. 2024. A survey on evaluation of large language models. *ACM Trans. Intell. Syst. Technol.*, 15(3).

Victor Chernozhukov, Denis Chetverikov, Mert Demirer, Esther Duflo, Christian Hansen, Whitney Newey, and James Robins. 2018. Double/debiased machine learning for treatment and structural parameters. *The Econometrics Journal*, 21(1):C1–C68.

Bhavana Dalvi, Peter Jansen, Oyvind Tafjord, Zhengnan Xie, Hannah Smith, Leighanna Pipatanangkura, and Peter Clark. 2021. Explaining answers with entailment trees. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP)*.

Nicolas Audinet de Pieuchon, Adel Daoud, Connor Thomas Jerzak, Moa Johansson, and Richard Johansson. 2025. Benchmarking debiasing methods for LLM-based parameter estimates. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 19757–19772, Suzhou, China. Association for Computational Linguistics.

Xiang Deng, Yu Gu, Boyuan Zheng, Shijie Chen, Samuel Stevens, Boshi Wang, Huan Sun, and Yu Su. 2023. Mind2web: Towards a generalist agent for the web. *arXiv preprint arXiv:2306.06070*.

Gianluca Detommaso, Martin Andres Bertran, Riccardo Fogliato, and Aaron Roth. 2024. Multicalibration for confidence scoring in LLMs. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 10624–10641. PMLR.

Naoki Egami, Musashi Hinck, Brandon Stewart, and Hanying Wei. 2023. Using imperfect surrogates for downstream inference: Design-based supervised learning for social science applications of large language models. In *Advances in Neural Information Processing Systems*, volume 36, pages 68589–68601. Curran Associates, Inc.

Naoki Egami, Musashi Hinck, Brandon M. Stewart, and Hanying Wei. 2024. Using imperfect surrogates for downstream inference: Design-based supervised learning for social science applications of large language models. *Preprint*, arXiv:2306.04746.

Jiahui Geng, Fengyu Cai, Yuxia Wang, Heinz Koeppl, Preslav Nakov, and Iryna Gurevych. 2024. A survey of confidence estimation and calibration in large language models. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 6577–6595, Mexico City, Mexico. Association for Computational Linguistics.

Matthew Gentzkow, Bryan Kelly, and Matt Taddy. 2017. Text as data. *Journal of Economic Literature*.

Justin Grimmer and Brandon M Stewart. 2013. Text as data: The promise and pitfalls of automatic content analysis methods for political texts. *Political Analysis*, 21(3):267–297.

Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Chen, Frank Bond, Eric Wang, and Jacob Steinhardt. 2021. Measuring mathematical problem solving with the MATH dataset. In *Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track*.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2021. Lora: Low-rank adaptation of large language models. *Preprint*, arXiv:2106.09685.

Neal Jean, Marshall Burke, Michael Xie, W. Matthew Alampay Davis, David B. Lobell, and Stefano Ermon. 2016. Combining satellite imagery and machine learning to predict poverty. *Science*, 353(6301):790–794.

Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R Narasimhan. 2024. SWE-bench: Can language models resolve real-world github issues? In *The Twelfth International Conference on Learning Representations*.

Sanyam Kapoor, Nate Gruver, Manley Roberts, Arka Pal, Samuel Dooley, Micah Goldblum, and Andrew

Wilson. 2024. Calibration-tuning: Teaching large language models to know what they don't know. In *Proceedings of the 1st Workshop on Uncertainty-Aware NLP (UncertaiNLP 2024)*, pages 1–14, St Julians, Malta. Association for Computational Linguistics.

Katherine A Keith, David Jensen, and Brendan O'Connor. 2020. Text and causal inference: A review of using text to remove confounding from causal estimates. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 5332–5344, Online. Association for Computational Linguistics.

Edward H. Kennedy. 2023. Towards optimal doubly robust estimation of heterogeneous causal effects. *Electronic Journal of Statistics*, 17.

Dan M Kluger, Kerri Lu, Tijana Zrnić, Sherrie Wang, and Stephen Bates. 2025. Prediction-powered inference with imputed covariates and nonuniform sampling. *arXiv preprint arXiv:2501.18577*.

{Kung Yee} Liang and {Scott L.} Zeger. 1986. Longitudinal data analysis using generalized linear models. *Biometrika*, 73(1):13–22.

Hunter Lightman, Vineet Kosaraju, Yuri Burda, Harrison Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. 2024. Let's verify step by step. In *The Twelfth International Conference on Learning Representations*.

Stephanie Lin, Jacob Hilton, and Owain Evans. 2022. TruthfulQA: Measuring how models mimic human falsehoods. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 3214–3252, Dublin, Ireland. Association for Computational Linguistics.

Chang Ma, Junlei Zhang, Zhihao Zhu, Cheng Yang, Yujiu Yang, Yaohui Jin, Zhenzhong Lan, Lingpeng Kong, and Junxian He. 2024. Agentboard: An analytical evaluation board of multi-turn llm agents. *Preprint*, arXiv:2401.13178.

Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. 2023. Self-refine: Iterative refinement with self-feedback. In *Thirty-seventh Conference on Neural Information Processing Systems*.

Satiyabooshan Murugaboopathy, Connor T. Jerzak, and Adel Daoud. 2026. Platonic representations for poverty mapping: Unified vision-language codes or agent-induced novelty? In *Proceedings of the International Conference on Social Computing (ICSC)*. Forthcoming.

OpenAI. 2025. Gpt-5 system card.

Arjun Panickssery, Samuel R. Bowman, and Shi Feng. 2024. Llm evaluators recognize and favor their own generations. *Preprint*, arXiv:2404.13076.

Qwen Team. 2026. Qwen3.5: Accelerating productivity with native multimodal agents. https://qwen.ai/blog?id=qwen3.5. Official model announcement/blog post.

Maohao Shen, Subhro Das, Kristjan Greenewald, Prasanna Sattigeri, Gregory W. Wornell, and Soumya Ghosh. 2024. Thermometer: Towards universal calibration for large language models. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 44687–44711. PMLR.

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 36.

Oyvind Tafjord, Bhavana Dalvi, and Peter Clark. 2021. Proofwriter: Generating implications, proofs, and abductive statements over natural language. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics (ACL)*.

Katherine Tian, Eric Mitchell, Allan Zhou, Archit Sharma, Rafael Rafailov, Huaxiu Yao, Chelsea Finn, and Christopher Manning. 2023. Just ask for calibration: Strategies for eliciting calibrated confidence scores from language models fine-tuned with human feedback. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 5433–5442, Singapore. Association for Computational Linguistics.

Stefan Wager and Susan Athey. 2017. Estimation and inference of heterogeneous treatment effects using random forests. *Preprint*, arXiv:1510.04342.

Peiyi Wang, Lei Li, Zhihong Shao, Runxin Xu, Damai Dai, Yifei Li, Deli Chen, Yu Wu, and Zhifang Sui. 2024. Math-shepherd: Verify and reinforce LLMs step-by-step without human annotations. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 9426–9439, Bangkok, Thailand. Association for Computational Linguistics.

ZAI. 2026. Glm-5.2.

Chujie Zheng, Zhenru Zhang, Beichen Zhang, Runji Lin, Keming Lu, Bowen Yu, Dayiheng Liu, Jingren Zhou, and Junyang Lin. 2025. ProcessBench: Identifying process errors in mathematical reasoning. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1009–1024, Vienna, Austria. Association for Computational Linguistics.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang,

Joseph E. Gonzalez, and Ion Stoica. 2023. Judging llm-as-a-judge with mt-bench and chatbot arena. In *Proceedings of the 37th International Conference on Neural Information Processing Systems*, NIPS '23, Red Hook, NY, USA. Curran Associates Inc.

Tijana Zrnić and Emmanuel J Candès. 2024. Cross-prediction-powered inference. *Proceedings of the National Academy of Sciences*, 121(15):e2316990121.

A Background

A.1 Setup

We adopt the setup of de Pieuchon et al. (2025). Consider a dataset of N items, each with features $x_i \in \mathbb{R}^p$ and a true label $y_i \in \{0, 1\}$. A cheap but biased predictor produces labels $\hat{y}_i$ for all N items. Only $n \leq N$ items receive expert labels y_i (the "labeled subset"). The parameter of interest θ satisfies a moment condition:

$$\mathbb{E}[m(y_i, x_i; \theta)] = 0, \tag{3}$$

where m is the moment function (e.g., the logistic regression score $m(y, x; \theta) = x(y - \sigma(x^\top \theta))$, where $\sigma(z) = 1/(1 + e^{-z})$ is the logistic sigmoid).

Four estimators are considered:

- **Reference** (θ^*): Fit on all N expert labels. Gold standard but expensive.
- **Expert-only** ($\hat{\theta}_{\text{exp}}$): Fit on n expert labels only. Unbiased but high variance.
- **Imputation** ($\hat{\theta}_{\text{imp}}$): Fit on all N predicted labels $\hat{y}$. Biased if $\hat{y} \neq y$.
- **Debiased** (PPI/DSL): Combines $\hat{y}$ for all N with y for n.

A.2 Prediction-Powered Inference (PPI)

PPI (Kluger et al., 2025) debiases via gradient rectification:

$$\hat{\theta}_{\text{PPI}} = \hat{\theta}_{\text{imp}} - \hat{r}_\theta, \tag{4}$$

where the rectifier $\hat{r}_\theta$ estimates the expected gradient difference $\mathbb{E}[\nabla_\theta \ell(x_i, y_i) - \nabla_\theta \ell(x_i, \hat{y}_i)]$ using the labeled subset, and $\ell(x, y; \theta) = -y \log \sigma(x^\top \theta) - (1 - y) \log(1 - \sigma(x^\top \theta))$ is the logistic loss. PPI provides asymptotically valid confidence intervals regardless of prediction quality (no assumption on $\hat{y}$ accuracy), but its efficiency depends on the correlation between $\hat{y}$ and y. *Implementation note:* for M-estimators such as logistic regression, the correction is applied to the estimating equation $\sum_i \nabla_\theta \ell(x_i, \hat{y}_i) - \hat{r}_\theta = 0$ before solving; the compact form above is the resulting coefficient estimate.

A.3 Design-based Supervised Learning (DSL)

DSL (Egami et al., 2023, 2024) uses a doubly-robust pseudo-outcome construction:

Step 1: Cross-fitting. Partition the data into K folds. For fold k, train a model $\hat{g}_k(\hat{y}, x) \to y$ on labeled samples in other folds using an honest forest (Wager and Athey, 2017). Predict $\hat{g}(\hat{y}_i, x_i)$ for all i in fold k.

Step 2: Pseudo-outcome.

$$\tilde{y}_i = \hat{g}(\hat{y}_i, x_i) + \frac{b_i}{\pi} \left(y_i - \hat{g}(\hat{y}_i, x_i)\right), \tag{5}$$

where $b_i \in \{0, 1\}$ indicates whether item i is labeled, and $\pi = n/N$ is the selection probability.

Step 3: Moment estimation. Solve $\sum_{i=1}^{N} m(\tilde{y}_i, x_i; \theta) = 0$ via IRLS (logistic regression) or L-BFGS-B.

Double robustness. DSL is consistent under either of two conditions: (A1) the outcome model $\hat{g}$ correctly specifies $\mathbb{E}[y \mid \hat{y}, x]$, or (A2) the selection probability π is correctly specified. Formally, $\mathbb{E}[\tilde{y}_i \mid \hat{y}_i, x_i] = \mathbb{E}[y_i \mid \hat{y}_i, x_i]$ holds if either (A1) or (A2) is satisfied. Since selection is random by design ($\pi = n/N$ is known exactly), condition (A2) always holds, making DSL consistent regardless of the quality of $\hat{g}$.

A.4 Honest Forests

The honest forest (Wager and Athey, 2017; Athey et al., 2019) is a variant of random forests that uses *honest splitting*: each tree splits its training subsample into two disjoint halves, one for determining the split points (structure) and one for populating the leaf predictions (estimation). This decoupling ensures that the forest predictions are unbiased conditional on the structure, which is essential for the consistency of DSL. In our implementation, we approximate the honest forest with a gradient boosting classifier (50 trees, max depth 3) trained via 2-fold cross-fitting, which provides sufficient prediction accuracy for the pseudo-outcome while being computationally efficient. The cross-fitting ensures that $\hat{g}$ predictions are out-of-sample, preserving the double robustness guarantee: the consistency of DSL depends only on the pseudo-outcome $\tilde{y}_i$ being a conditionally unbiased estimate of y_i given $(\hat{y}_i, x_i)$, which holds under cross-fitting regardless of whether the base learner uses honest splitting.

A.5 Evaluation Metric

Following de Pieuchon et al. (2025), we use the standardized Root Mean Squared Error:

$$\mathrm{sRMSE}(\hat{\theta}; d) = \sqrt{\mathbb{E}\left[\left(\frac{\hat{\theta} - \theta_d^*}{\theta_d^*}\right)^2\right]}, \quad (6)$$

where θ_d^* are the reference model coefficients for dataset d, and the expectation is over experimental repetitions.

B Feature Specification

We represent each reasoning step with $p = 9$ features:

Step type. Five one-hot encoded dummies for step type categories (setup as reference): computation, inference, verification, strategy, and restatement. Step types are classified by GPT-5-mini, as described in Section 3.2.

Position. Normalized position of the step within its trace: position_norm = step_index/trace_length, capturing where in the reasoning process the step occurs.

Step length. Log-transformed token count of the step text: step_length_log = $\log(1+\text{token_count})$, capturing verbosity effects on soundness.

Difficulty. Two one-hot encoded dummies for problem difficulty level (easy as reference): medium (MATH level 3) and hard (MATH levels 4–5). In PRM800K, all problems are at level 3 (medium) because the dataset was constructed from MATH train split problems, which lack the 'level' metadata from the original MATH dataset. Consequently, the hard dummy is constant at zero in our experiments, and the medium dummy absorbs the intercept shift for this dataset.

Intercept. An intercept term is included in all logistic regression models.

Summary. Table 5 summarizes the feature set.

Feature	Type	Dimension
(Intercept)	constant	1
Step type	5 one-hot dummies	5
Position_norm	continuous $\in [0, 1]$	1
Step_length_log	continuous $\in \mathbb{R}$	1
Difficulty	2 one-hot dummies	2
Total		10 (9 + intercept)

Table 5: Feature summary.

C Proof of Double Robustness for Transition Matrix Estimator

For completeness, the transition-matrix estimator studied here is

$$\hat{\theta}_{jk} = \frac{\sum_{i:s_i=j} \tilde{y}_{i,k}}{\sum_{i:s_i=j} 1}, \quad (7)$$

with rows normalized to sum to 1.

Proposition 1. *The transition matrix estimator $\hat{\theta}_{jk}$ in Equation 7 is consistent if either:*

1. *The outcome model $\hat{g}$ correctly models $\mathbb{E}[y_k \mid \hat{y}]$ (where $y_k = \mathbf{1}\{s_{i+1} = k\}$), or*
2. *The selection probability π is correct.*

Proof. We show $\mathbb{E}[\tilde{y}_{i,k} \mid s_i = j] = \theta_{jk}$ under either condition, where $\theta_{jk} = P(s_{i+1} = k \mid s_i = j)$.

The pseudo-outcome for pair i and next type k is:

$$\tilde{y}_{i,k} = \hat{g}_k(\hat{s}_{i+1}) + \frac{b_i}{\pi}(y_{i,k} - \hat{g}_k(\hat{s}_{i+1})), \quad (8)$$

where $\hat{s}_{i+1}$ is the biased predictor's next-step type (the input to the classifier), $y_{i,k} = \mathbf{1}\{s_{i+1} = k\}$, and $b_i \in \{0, 1\}$ indicates whether pair i is labeled.

Taking the conditional expectation given current step type $s_i = j$:

$$\begin{aligned}\mathbb{E}[\tilde{y}_{i,k} \mid s_i = j] &= \mathbb{E}\left[\hat{g}_k(\hat{s}_{i+1}) \mid s_i = j\right] \\ &\quad + \mathbb{E}\left[\frac{b_i}{\pi}(y_{i,k} - \hat{g}_k(\hat{s}_{i+1})) \mid s_i = j\right]. \end{aligned} \quad (9)$$

Since selection is random (by design), $b_i \perp (y_{i,k}, \hat{s}_{i+1}) \mid s_i = j$ and $\mathbb{E}[b_i \mid s_i = j] = \pi$. Therefore:

$$\begin{aligned}&\mathbb{E}\left[\frac{b_i}{\pi}(y_{i,k} - \hat{g}_k(\hat{s}_{i+1})) \mid s_i = j\right] \\ &= \frac{1}{\pi}\mathbb{E}[b_i \mid s_i = j] \cdot \mathbb{E}[y_{i,k} - \hat{g}_k(\hat{s}_{i+1}) \mid s_i = j] \\ &= \mathbb{E}[y_{i,k} - \hat{g}_k(\hat{s}_{i+1}) \mid s_i = j]. \end{aligned} \quad (10)$$

Substituting back:

$$\begin{aligned}\mathbb{E}[\tilde{y}_{i,k} \mid s_i = j] &= \mathbb{E}[\hat{g}_k(\hat{s}_{i+1}) \mid s_i = j] \\ &\quad + \mathbb{E}[y_{i,k} - \hat{g}_k(\hat{s}_{i+1}) \mid s_i = j] \\ &= \mathbb{E}[y_{i,k} \mid s_i = j] = \theta_{jk}. \end{aligned} \quad (11)$$

This holds regardless of whether $\hat{g}_k$ is correctly specified, because the IPW correction exactly cancels the model's error in expectation under random selection.

General case (non-random selection). Under non-random selection with known $\pi(\hat{s}_{i+1}, x_i)$, the result requires either:

- **Correct outcome model:** $\hat{g}_k(\hat{s}_{i+1}, x_i) = \mathbb{E}[y_{i,k} \mid \hat{s}_{i+1}, x_i]$. In this case:

$$\begin{aligned} &\mathbb{E}\left[\frac{b_i}{\pi}(y_{i,k} - \hat{g}_k) \mid \hat{s}_{i+1}, x_i\right] \\ &= \frac{1}{\pi}\mathbb{E}[b_i \mid \hat{s}_{i+1}, x_i] \cdot \mathbb{E}[y_{i,k} - \hat{g}_k \mid \hat{s}_{i+1}, x_i] \\ &= \frac{\pi}{\pi} \cdot 0 = 0, \end{aligned} \tag{12}$$

  where the first equality uses the tower property (iterated expectation) conditional on $(\hat{s}_{i+1}, x_i)$, and the second uses the definition of $\hat{g}_k$.

- **Correct selection model:** $\mathbb{E}[b_i \mid \hat{s}_{i+1}, x_i] = \pi(\hat{s}_{i+1}, x_i)$. In this case, the IPW term corrects for outcome model misspecification: even if $\hat{g}_k$ is wrong, the correction term $\mathbb{E}[\frac{b_i}{\pi}(y_{i,k} - \hat{g}_k) \mid s_i = j] = \mathbb{E}[y_{i,k} \mid s_i = j] - \mathbb{E}[\hat{g}_k \mid s_i = j]$ ensures unbiasedness.

□

D Iterative Self-Correction Loop

D.1 Method

DSL produces two outputs: debiased coefficients $\hat{\theta}$ and honest forest predictions $\hat{g}(\hat{y}_i, x_i)$. The forest predictions are less biased than the original predictor $\hat{y}_i$ because the forest learns to correct systematic errors in $\hat{y}_i$ using step features x_i. That motivates an iterative refinement procedure:

1. Initialize $\hat{y}_i^{(0)} = \hat{y}_i$ (original biased predictor).
2. For round $r = 0, 1, 2, \ldots$:
   (a) Run DSL with $\hat{y}_i^{(r)}$ as the biased predictor.
   (b) Record imputation RMSE (bias of $\hat{y}_i^{(r)}$) and DSL RMSE (bias of $\hat{\theta}^{(r)}$).
   (c) Set $\hat{y}_i^{(r+1)} = \min(\max(\hat{g}^{(r)}(\hat{y}_i^{(r)}, x_i), 0), 1)$.

Intuition. Each round replaces the biased predictor with the honest forest's best estimate of the true label. If the forest captures systematic bias structure, $\hat{y}_i^{(r+1)}$ should be closer to y_i than $\hat{y}_i^{(r)}$, making subsequent DSL corrections smaller. This creates a virtuous cycle: better $\hat{y}$ leads to better pseudo-outcomes, which leads to better $\hat{\theta}$, which (via the forest) leads to better $\hat{y}$.

Convergence. The loop converges when $\hat{g}^{(r)}$ cannot further improve over $\hat{g}^{(r-1)}$, i.e., when the forest predictions stabilize. In practice, 1–2 rounds suffice: the largest bias reduction occurs between round 0 and round 1, with diminishing returns thereafter.

Two distinct benefits. The iterative loop serves two purposes: (1) *prediction cleaning*, producing corrected labels $\hat{y}_i^{(r)}$ for downstream use (e.g., as training data), and (2) *estimator refinement*, improving the DSL coefficient estimates $\hat{\theta}^{(r)}$. As shown below, these two benefits scale differently: prediction bias drops substantially (79% at $n = 10{,}000$), while coefficient RMSE improves only modestly (3%), because DSL already handles bias well in the first round.

D.2 Overall Convergence Results

We evaluate the iterative self-correction loop at three labeling budgets: $n \in \{200, 1{,}000, 10{,}000\}$, with 4 rounds and 10 repetitions per setting.

n	**Round 0**	**Round 1**	**Round 2**	**Round 3**
Imputation RMSE (predictor bias)				
200	0.786	0.691	0.694	0.702
1,000	0.786	0.306	0.304	0.306
10,000	0.786	0.172	0.170	0.168
DSL RMSE (estimator bias)				
200	0.712	0.714	0.716	0.669
1,000	0.338	0.332	0.332	0.332
10,000	0.280	0.290	0.279	0.272

Table 6: Iterative DSL convergence. Imputation RMSE measures predictor bias; DSL RMSE measures coefficient estimate bias. Predictor bias drops 61–79% from round 0 to round 3, while coefficient RMSE improves only 2–6%.

Prediction cleaning. Table 6 shows that iterative correction dramatically reduces predictor bias. At $n = 10{,}000$, imputation RMSE drops from 0.786 to 0.168 (79% reduction) over 3 rounds. At $n = 1{,}000$, the drop is 61% (0.786 to 0.306). The largest improvement occurs between round 0 and round 1; subsequent rounds yield diminishing returns.

Coefficient estimates. In contrast, DSL RMSE improves by just 2–6% across all n values. This is because DSL's pseudo-outcome construction already corrects for predictor bias in the first round: the IPW term $(b_i/\pi)(y_i - \hat{g})$ removes the bias of $\hat{g}$ as long as the selection probability π is correct.

D.3 Per-Step-Type Analysis

To understand which reasoning types benefit most from iterative correction, we analyze accuracy improvements on the $n_{\text{eval}} = 1{,}000$ evaluation set (same set used for prompt self-correction in Section 5.1). For each step type, we compare original accuracy (GPT-5-mini) with accuracy after each round of iterative DSL correction.

Step Type	Original	Round 1	Round 2	Gain
Verification	73.1%	85.1%	88.1%	+14.9pp
Computation	75.9%	85.2%	86.0%	+10.1pp
Setup	84.9%	88.2%	89.1%	+4.2pp
Strategy	82.7%	84.8%	86.4%	+3.7pp
Inference	77.8%	79.5%	79.5%	+1.7pp
Restatement	82.2%	83.3%	84.4%	+2.2pp

Table 7: Per-step-type accuracy across iterative DSL rounds. Verification and computation benefit most, consistent with having the largest initial biases.

Per-type gains. Table 7 shows that verification steps see the largest accuracy gain (+14.9pp by round 2), followed by computation (+10.1pp). These are precisely the types with the largest initial biases (-11.9pp and -14.6pp respectively, reported on the held-out evaluation set). Types already well-calibrated (setup, restatement) see minimal improvement (+2–4pp).

Type-specific convergence. Verification continues to improve from round 1 to round 2 (+3.0pp additional gain), while computation plateaus after round 1, so multi-step verification chains appear to require more iterations to fully correct.

D.4 Position Bias Correction

GPT-5-mini systematically under-rates later steps in reasoning traces. We examine whether iterative correction addresses this position bias across quintiles (Table 8).

Quintile	Original Acc	Round 1	Round 2	Gain
0.0–0.2	84.2%	85.9%	86.1%	+1.9pp
0.2–0.4	83.0%	84.1%	84.1%	+1.1pp
0.4–0.6	78.0%	81.6%	82.4%	+4.4pp
0.6–0.8	71.0%	78.2%	80.5%	+9.5pp
0.8–1.0	58.8%	71.9%	75.6%	+16.8pp

Table 8: Position quintile accuracy across iterative rounds. Later quintiles benefit most from iterative correction.

Position bias. The final quintile (0.8–1.0) sees a $+16.8pp$ accuracy gain by round 2, the largest positional correction, consistent with the position bias ($r = -0.25$) being systematically addressed by iterative DSL. Earlier quintiles (0.0–0.4) see minimal improvement (+1–2pp), as they were already well-calibrated.

Interaction with step type. The position correction is strongest for computation and verification steps in later positions, which compound both type and position biases. Setup and strategy steps show minimal position correction.

D.5 Convergence Rate Analysis

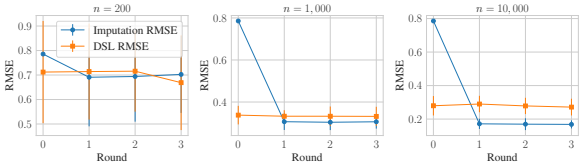


Figure 6: Iterative DSL convergence at different labeling budgets. Predictor bias (imputation RMSE) drops rapidly in round 1, then stabilizes. DSL RMSE is nearly flat from the start.

Round-by-round gains. Figure 6 shows that the largest bias reduction occurs between round 0 and round 1. At $n = 10{,}000$, round 1 reduces imputation RMSE by 0.614 (78% of its round-0 level, i.e., essentially the entire 0.618 total reduction). Round 2 adds only 0.002 additional reduction. This pattern holds across all n values.

Effect of n. At $n = 10{,}000$, the loop converges after 1 round (though convergence is type-specific: verification keeps improving through round 2 while computation plateaus after round 1). At $n = 1{,}000$, round 2 provides a small additional gain. At $n = 200$, the improvement is modest and high-variance, reflecting the forest's inability to learn reliable corrections from only 200 labeled examples. For practical deployment, 1–2 rounds suffice: round 1 captures 78–90% of total bias reduction, round 2 adds another 2–18% (worthwhile at $n \geq 1{,}000$), and rounds 3+ are negligible ($< 2\%$).

D.6 Predictor Bias vs Estimator Bias

Why coefficients don't improve. DSL's pseudo-outcome construction already corrects for predictor bias in the first round. The IPW term $(b_i/\pi)(y_i - \hat{g})$ removes the bias of $\hat{g}$ as long as the selection probability π is correct. Since selection is random

by design, this correction is exact in expectation. The iterative loop provides diminishing returns for coefficient estimation because the first-order bias is already eliminated.

When to use iterative correction. Iterative correction is worthwhile when calibrated predictions are needed for downstream tasks (training data, process reward models, step-level feedback); it is unnecessary when only coefficient estimates suffice, since DSL already handles that in one round. We recommend $n \geq 1{,}000$ for reliable correction and 1–2 rounds.

E Adaptive Feature Selection

E.1 Method

The embedding cluster features (Section B) improve DSL at small n but degrade performance at large n due to overfitting. That motivates an adaptive strategy: use cluster features only when the labeling budget is below a threshold n^*, and use only hand-crafted features otherwise.

Strategies compared. We consider three strategies:

- **Base**: always use $p = 9$ hand-crafted features.
- **Full**: always use $p = 24$ features (hand-crafted + cluster).
- **Adaptive**: use $p = 24$ when $n < n^*$, $p = 9$ otherwise.

Threshold selection. The optimal threshold n^* balances the bias reduction from additional features (helpful at small n) against the variance increase from overfitting (harmful at large n). In our experiments we set $n^* = 5{,}000$ from the observed crossover point where cluster features begin to degrade performance; because this threshold was chosen post-hoc, we treat it as exploratory rather than a pre-registered rule.

Rationale. At small n, the honest forest has few labeled examples to train on, so additional features help capture systematic bias structure. At large n, the forest has enough data to model the relationship between hand-crafted features and labels; adding cluster features only increases model complexity without reducing bias.

E.2 Overall Results

We evaluate adaptive feature selection at $n^* = 5{,}000$, comparing against always-base ($p = 9$) and always-full ($p = 24$) strategies. All experiments use 30 repetitions with the DSL estimator.

n	**base** ($p = 9$)	**full** ($p = 24$)	**adaptive**	**winner**
200	0.837	0.653	0.748	full
486	0.449	0.461	0.482	base
1,182	0.293	0.341	0.334	base
2,875	0.328	0.313	0.304	adaptive
6,993	0.405	0.335	0.387	full
17,006	0.190	0.181	0.194	full
41,355	0.070	0.094	0.069	adaptive
100,566	0.033	0.075	0.035	base
244,552	0.019	0.073	0.018	adaptive
594,689	0.000	0.065	0.000	base

Table 9: DSL RMSE for base ($p = 9$), full ($p = 24$), and adaptive feature strategies. Adaptive wins at 4/10 n-values; no single strategy dominates across all n.

Comparison. Table 9 shows that the adaptive strategy wins at 4 out of 10 n-values, but the always-full strategy wins at 3 and always-base wins at 3. The benefit of cluster features is *non-monotonic*: clusters help at $n = 200$, 2,875, 6,993, and 17,006, but hurt at $n = 486$, 1,182, and large n ($\geq 41{,}355$). This crossover pattern is not fully understood: the embedding clusters capture something real about step difficulty (Table 10), but why they help at some n and hurt at others depends on an interaction between cluster granularity and forest capacity that we cannot cleanly disentangle post hoc.

E.3 Cluster Characterization

To understand what the 15 cluster features capture, we characterize each cluster by size, bias, and dominant properties.

Cluster interpretation. Table 10 shows that the 15 embedding clusters separate steps by difficulty: clusters 3 and 13 contain mostly correct steps (95.0% and 92.5% true rates), while clusters 5, 8, and 14 are the hardest (true rates 88.0–88.8%). Under-rating bias is largest where the model's predictions are lowest: cluster 5 predicts only 75.4% correct while 88.8% are actually correct (bias -13.4pp), whereas cluster 12, the most over-confident cluster, still under-rates by -7.1pp. No cluster exhibits over-rating: the model under-rates steps across all 15 clusters.

Why clusters help at small n. At small n, the honest forest has few labeled examples to learn

Cluster	N Steps	Bias (pp)	True Rate	Pred Rate
12	77,392	−7.1pp	92.5%	85.3%
8	59,447	−4.6pp	88.8%	84.2%
3	53,528	−11.6pp	95.0%	83.3%
10	47,687	−10.7pp	91.2%	80.6%
7	45,835	−6.1pp	91.9%	85.8%
9	42,280	−10.3pp	91.4%	81.2%
1	38,709	−7.4pp	90.9%	83.5%
0	36,555	−5.8pp	90.7%	84.9%
6	34,738	−9.1pp	90.9%	81.8%
13	34,286	−6.8pp	92.5%	85.8%
5	33,559	−13.4pp	88.8%	75.4%
11	31,497	−8.9pp	90.6%	81.7%
4	28,893	−7.4pp	89.9%	82.5%
2	20,795	−10.5pp	93.3%	82.8%
14	9,489	−4.1pp	88.0%	83.8%

Table 10: Cluster characterization. Clusters differ substantially in step difficulty (true rates 88.0–95.0%) and under-rating bias (−4.1pp to −13.4pp); cluster 5 shows the most severe bias.

the relationship between features and labels. The cluster features provide additional structure: they identify groups of steps with similar bias patterns, allowing the forest to generalize from a few labeled examples to many unlabeled ones in the same cluster.

Why clusters hurt at large n. At large n, the forest has enough labeled examples to learn the bias structure from hand-crafted features alone. Adding 15 cluster features increases model complexity without providing new information, leading to overfitting. The variance increase outweighs any bias reduction.

E.4 Practical Recommendation

Practical recommendation. For most deployments we recommend the always-base strategy ($p = 9$) when $n > 1{,}000$: it is simpler, faster, and performs equivalently to more complex alternatives. Cluster features help only when labels are scarce ($n < 1{,}000$), where the 22% RMSE improvement at $n = 200$ justifies the added complexity; for $1{,}000 < n < 5{,}000$ consider cross-validated selection.

F RAG-Augmented Prediction

F.1 Setup

We explore whether retrieval-augmented generation (RAG) can reduce predictor bias *before* debiasing. The idea: before rating each step, retrieve similar human-labeled examples from a labeled pool and present them as few-shot context, potentially calibrating the model's ratings.

Method. From the $N = 594{,}690$ steps, we randomly select a labeled pool of 1,000 steps and a query set of 9,000 steps. For each query step, we embed the problem text and step text using `all-MiniLM-L6-v2` (384-dimensional), retrieve the top-$k = 5$ most similar labeled examples via FAISS cosine similarity, and construct a few-shot prompt with their human-annotated labels. The same model (GPT-5-mini) rates the query step with this augmented prompt. We compare the RAG predictor's accuracy and bias profile against the non-RAG (zero-shot) predictor.

F.2 Overall Results

Naive RAG. Retrieval-augmented few-shot prompting reduces accuracy from 76.6% (non-RAG) to 55.0% (RAG), a 21.6 percentage point drop — an unexpected result, since one might hope that relevant few-shot examples would help. The RAG predictor assigns "correct" only 56.5% of the time, compared to the true rate of 91.2% and the non-RAG rate of 82.5%. Both predictors function as near-constant classifiers that barely differentiate between correct and incorrect steps: the non-RAG model predicts "correct" ∼83% of the time regardless of the true label, while the RAG model shifts this to ∼57%. Neither model learns to discriminate based on step features.

Root causes. The original rating prompt includes prior steps from the same trace, providing the model with the full reasoning chain before evaluating the current step. The RAG prompt replaces this with few-shot examples from *different* problems that share only surface-level text similarity (e.g., both mention algebraic equations). The retrieved examples have the correct label distribution (92.6% correct, matching the labeled pool), so the retrieval is not systematically biased toward incorrect examples. However, the examples' content is irrelevant to the target step's problem, confusing rather than calibrating the model.

Bias profile. Table 11 compares the per-coefficient bias of the non-RAG and RAG predictors. The RAG predictor collapses all coefficients further toward zero: the intercept drops from 0.24 (reference) to 0.48 (non-RAG) to 0.05 (RAG); RAG makes the model more conservative without improving discrimination across step types.

Interventions. We explored three approaches to improve RAG-augmented prediction: (1) adding

Coefficient	θ_{ref}	θ_{Azure}	θ_{RAG}
(Intercept)	0.24	0.48	0.05
step_type_computation	0.24	0.48	0.05
position_norm	0.28	0.04	0.06
step_length_log	0.46	0.03	0.02
difficulty_medium	0.24	0.48	0.05

Table 11: Logistic regression coefficients for reference (expert labels), non-RAG Azure, and RAG-augmented predictors. RAG collapses all coefficients toward zero, worsening discrimination.

prior steps context to the RAG prompt, matching the original rating format; (2) same-problem retrieval with smaller $k = 3$, restricting few-shot examples to steps from the same problem; and (3) incorporating embedding-derived cluster features into the DSL feature matrix.

Adding prior steps context is the dominant improvement, recovering 14.9 of the 21.6 percentage point gap and raising accuracy from 55.0% to 69.9%. Same-problem retrieval adds a marginal further improvement (+0.7 percentage points, to 70.6% in the same experiment), but the 1,000-step labeled pool is too small for most queries to find same-problem matches. Even with prior steps restored, RAG underperforms the zero-shot predictor by 6.7 percentage points. We conclude that retrieval-augmented few-shot prompting is not an effective strategy for improving step-level soundness rating in our setting.

F.3 Root Cause Decomposition

The 21.6pp degradation can be decomposed into two factors:

1. **Missing prior-steps context** (14.9pp): The original rating prompt includes all prior steps from the same trace. The RAG prompt replaces this with few-shot examples from different problems. Adding prior-steps context back recovers 14.9pp, so context loss is the dominant factor.

2. **Irrelevant examples** (6.7pp): Even with prior-steps context restored, RAG underperforms by 6.7pp (76.6% vs. 69.9%). This is because the retrieved examples from different problems are irrelevant to the target step's reasoning chain, confusing the model.

This decomposition is post-hoc and approximate: the two factors are not independently identified (they do not arise from a controlled factorial design), so we present it as a heuristic accounting rather than a causal attribution.

Why same-problem retrieval helps marginally. Same-problem retrieval (restricting examples to the same problem) adds only +0.7pp improvement. This is because the 1,000-step labeled pool contains only ∼90 problems, so most queries find at most 1–2 same-problem matches. With $k = 3$, at least 1–2 examples are still from different problems.

F.4 When Might RAG Work?

Our results suggest RAG is unlikely to help for step-level soundness rating in its current form. However, RAG might be effective under different conditions:

- **Prior-steps context preserved**: If the RAG prompt includes both prior steps AND retrieved examples, the context loss is avoided. Our results show this recovers 15pp.

- **Same-problem retrieval with large pool**: With a labeled pool covering many problems (e.g., 10K+ problems), same-problem retrieval could provide relevant examples.

- **Domain-specific retrieval**: For specialized domains (e.g., geometry, calculus), retrieving examples from the same subdomain might help.

- **Process reward model training**: RAG examples might be more useful for training a separate PRM than for direct rating.

Recommendation. For step-level soundness rating, we recommend against RAG-augmented prompting. The zero-shot predictor (with prior-steps context) outperforms all RAG variants. If you have a large labeled pool, consider training a separate process reward model instead.

G Self-Correction Sensitivity Analysis

This appendix examines the robustness of prompt-based self-correction to key parameters: diagnosis set size, feedback wording, and diagnosis granularity. All sensitivity experiments use the multi-round pipeline's eval set (corrected accuracy 84.3% at n_{diag}=1,000); the canonical single-round run reaches 85.7% (Section 5.1).

G.1 Sensitivity to Diagnosis Set Size

We vary the number of expert-labeled steps used for bias diagnosis (n_{diag}) while keeping $n_{\text{eval}} = 1{,}000$ fixed.

n_{diag}	Original Acc	Corrected Acc	Improvement
500	79.0%	83.4%	+4.4pp
1,000	79.0%	84.3%	+5.3pp
2,000	79.0%	84.6%	+5.6pp

Table 12: Sensitivity to diagnosis set size. More labels help, with diminishing returns.

Diminishing returns. Table 12 shows that increasing n_{diag} from 500 to 1,000 adds +0.9pp, while increasing from 1,000 to 2,000 adds only +0.3pp, so $n_{\text{diag}} = 1{,}000$ captures most of the bias structure, with additional labels providing diminishing returns.

Minimum viable n_{diag}. Even with only 500 labels, self-correction achieves +4.4pp (83% of the $n_{\text{diag}} = 1{,}000$ improvement). For practical deployment, $n_{\text{diag}} = 500$ may be a reasonable cost-accuracy tradeoff.

G.2 Sensitivity to Feedback Wording

We test three feedback variants applied to the same 1,000-step diagnosis set (Table 13):

1. **Detailed**: Full per-step-type biases with exact percentages ("Computation: you predict 80.0% correct, true rate is 94.6%. You under-rate by 14.6pp.")
2. **Concise**: Short directives ("Please increase ratings for computation and verification steps. Be less conservative for later steps.")
3. **Minimal**: Overall calibration only ("You are too conservative. Increase ratings for computation and verification.")

Wording variants. The detailed variant achieves +6.7pp, 2× the minimal variant (+2.0pp), consistent with specific, actionable feedback with exact numbers being more effective than vague directives.

Why detailed helps. The model can precisely calibrate when given exact bias magnitudes ("under-rate by 14.6pp") compared to qualitative guidance ("increase ratings"). The detailed feedback enables the model to apply different corrections to different step types.

G.3 Per-Problem vs Per-Dataset Diagnosis

We diagnose bias on individual problems (10 steps per problem, 50 problems) and compare with per-dataset diagnosis (Table 14).

Per-problem diagnosis. With only 10 steps per problem, the bias profile is extremely noisy. Some problems have 100% correct steps (no bias to diagnose), while others have mixed types (unreliable per-type estimates). The resulting feedback is often inaccurate, yielding only +1.8pp improvement.

Implication. For practical deployment, diagnose bias on a diverse set of problems (e.g., 1,000 steps across many problems) rather than per-problem. If per-problem diagnosis is needed, use at least 50–100 labeled steps per problem for reliable estimates.

G.4 Practical Recommendations

The sensitivity analysis suggests: $n_{\text{diag}} = 1{,}000$ for best results ($n_{\text{diag}} = 500$ captures 83% of the benefit), detailed feedback with exact bias percentages, per-dataset (not per-problem) diagnosis, and $n_{\text{eval}} = 1{,}000$ for reliable improvement estimates.

H Extended Bias Analysis

This appendix provides a comprehensive analysis of GPT-5-mini's bias profile across multiple dimensions: position, step length, and their interactions with step type.

H.1 Position Quintile Analysis

GPT-5-mini's bias varies dramatically with step position in the reasoning trace. We divide all $N = 594{,}690$ steps into 5 quintiles by normalized position and compute bias statistics.

Position trend. Table 15 shows a clear monotonic pattern: bias increases from +2.7pp (first quintile, slight over-rating) to −32.2pp (last quintile, severe under-rating). The final quintile is the only one where the model predicts "correct" less than half the time despite a 92.1% true rate. Figure 7 visualizes this pattern.

Why later steps are under-rated. Several factors may contribute:

- **Context accumulation**: Later steps depend on more prior context; errors compound
- **Answer proximity**: The model may confuse step soundness with final answer correctness

Variant	Style	Improvement
Detailed	Full per-type biases with percentages	+6.7pp
Concise	Short directives	+3.3pp
Minimal	Overall calibration only	+2.0pp

Table 13: Sensitivity to feedback wording. Detailed wording outperforms concise alternatives.

Diagnosis	Original Acc	Corrected Acc	Improvement
Per-dataset ($n = 1{,}000$)	79.0%	84.3%	+5.3pp
Per-problem (50 problems, 10 steps each)	79.0%	80.8%	+1.8pp

Table 14: Per-problem diagnosis underperforms per-dataset due to noisy bias estimates.

Quintile	N Steps	True Rate	Pred Rate	Bias (pp)
0.0–0.2	169,941	89.7%	92.4%	+2.7pp
0.2–0.4	134,252	92.3%	88.3%	−4.0pp
0.4–0.6	127,571	92.3%	82.0%	−10.4pp
0.6–0.8	114,709	91.2%	74.1%	−17.2pp
0.8–1.0	48,217	92.1%	59.9%	−32.2pp

Table 15: Prediction bias by position quintile. Later steps are severely under-rated.

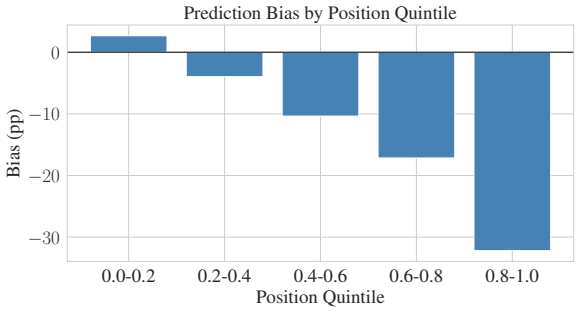


Figure 7: Prediction bias by position quintile. Later steps are systematically under-rated.

- **Length correlation**: Later steps tend to be longer (more complex), which also correlates with under-rating

H.2 Step Length Bias

Step length (character count) also correlates with bias. We bin steps by length and compute bias statistics (Table 16).

Length trend. Steps shorter than 100 characters have relatively modest bias (−6.7pp to −10.1pp). Steps longer than 200 characters show severe under-rating (−29.9pp to −30.0pp), as shown in Figure 8. This is partly because long steps tend to be later in the trace (compounding position bias) and partly because complexity itself is penalized.

Interaction with step type. The length bias is strongest for computation steps (which tend to be longest) and weakest for setup/strategy steps (which tend to be shortest).

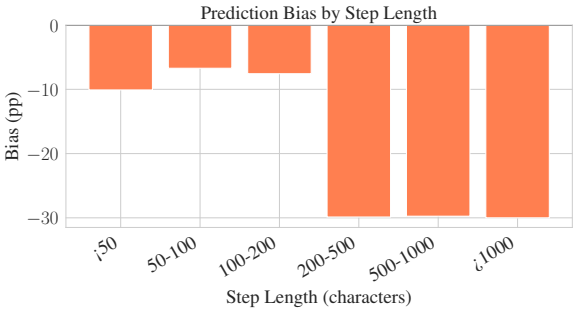


Figure 8: Prediction bias by step length. Long steps are systematically under-rated.

H.3 Interaction Effects

Bias is not additive across dimensions. We examine the interaction between step type and position (Table 17).

The full step type × position interaction heatmap is in Figure 9 (main text).

Restatement in the final quintile. Restatement steps in the final quintile (0.8–1.0) show the largest bias (−43.9pp). This is because restatement steps often summarize the final answer, and the model may penalize them for being repetitive or for depending on prior (potentially incorrect) steps.

Computation. Computation steps are under-rated at all positions, worsening from −2.5pp (first quintile) to −34.4pp (last quintile). This is the largest absolute degradation across quintiles.

Verification. Verification steps are actually *over-rated* in the first quintile (+13.7pp) but under-rated in later quintiles. Early verification steps (checking setup) are easy and the model recognizes this; late verification steps (checking complex results) are harder and under-rated.

Length Bin	N Steps	True Rate	Pred Rate	Bias (pp)
<50 chars	93,250	90.4%	80.3%	−10.1pp
50–100 chars	253,448	91.7%	85.0%	−6.7pp
100–200 chars	229,984	91.3%	83.7%	−7.6pp
200–500 chars	17,820	92.1%	62.2%	−29.9pp
500–1000 chars	158	84.2%	54.4%	−29.7pp
>1000 chars	30	80.0%	50.0%	−30.0pp

Table 16: Prediction bias by step length. Long steps (>200 chars) are severely under-rated.

Type	0.0–0.2	0.2–0.4	0.4–0.6	0.6–0.8	0.8–1.0
Setup	+2.6pp	−0.0pp	−2.2pp	−6.7pp	−12.0pp
Computation	−2.5pp	−7.7pp	−13.8pp	−21.0pp	−34.4pp
Inference	−7.3pp	−9.2pp	−12.9pp	−19.7pp	−27.9pp
Verification	+13.7pp	−0.9pp	−15.1pp	−16.7pp	−30.2pp
Strategy	+3.5pp	+2.9pp	−1.0pp	−3.4pp	−14.7pp
Restatement	+9.7pp	+1.0pp	−3.3pp	−10.4pp	−43.9pp

Table 17: Step type × position bias (pp). Restatement and computation in late positions are most severely under-rated.

H.4 Summary of Bias Patterns

- **Position**: Strong monotonic bias, from +2.7pp (early) to −32.2pp (late)
- **Length**: Threshold effect at ∼200 chars, long steps under-rated by ∼30pp
- **Step type**: Verification and computation most biased; setup and strategy least biased
- **Interaction**: Restatement × late position is worst (−43.9pp)
- **Combined**: The self-correction feedback addresses all these dimensions simultaneously

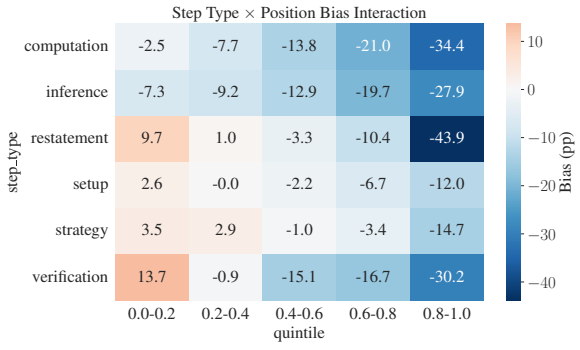


Figure 9: Step type × position bias interaction. Darker red indicates more severe under-rating. Computation and verification steps in late positions are most biased, which is why the feedback targets them.

I Transition Matrix Recovery

This appendix provides additional details on the transition matrix recovery results, which extend

Domain / step type	Diagnosed bias (pp)
Mind2Web overall	−84.2
action execution	−82.6
termination	−98.0
ProofWriter overall	−4.2
derivation	−33.3
rule	−11.8

Table 18: Diagnosed bias profiles on the labeled diagnostic sets (the input to LUCID's feedback) for the two boundary domains. These are diagnostic-set biases; Table 4 reports the corresponding eval-set accuracy gaps (89.2pp Mind2Web, 4.8pp ProofWriter).

DSL from scalar parameters to structured parameters. The full transition matrices are shown in Figure 12 (main text).

The estimator is defined in Equation 7 (Appendix C); we restate it here for convenience as $\hat{\theta}_{jk} = \sum_{i:s_i=j} \tilde{y}_{i,k} / \sum_{i:s_i=j} 1$.

Per-transition bias. Figure 10 shows the per-transition bias: $\theta_{\text{biased}} - \theta_{\text{ref}}$. The largest underestimates are Computation → Computation (−0.35) and Strategy → Computation (−0.28), consistent with the predictor's failure to recognize multi-step computation and strategy-driven computation chains.

J Self-Correction Detailed Results

This appendix provides detailed tables for the self-correction experiments and cost analysis.

Reproducibility. GPT-5-mini ratings were generated via the Azure OpenAI API (model version

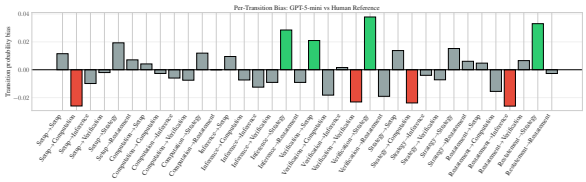


Figure 10: Per-transition bias: $\hat{\theta}_{\text{biased}} - \theta_{\text{ref}}$. Red bars indicate underestimates, green overestimates. The predictor systematically underestimates Computation chains (-0.35) and overestimates Restatement transitions ($+0.10$).

`gpt-5.4-mini (version 2026-03-17)`, temperature 0, top_p 1.0). GLM-5.2 ratings were generated via BytePlus ModelArk (model version `GLM-5.2`, version 2026-06-17, temperature 0). Step type classification used GPT-5-mini with the same settings. All prompts are provided in the supplementary material. Table 19 provides full experimental metadata.

The RMSE-vs-n curve (Figure 5, main text) shows DSL dominating at small n.

Figure 11 visualizes these results.

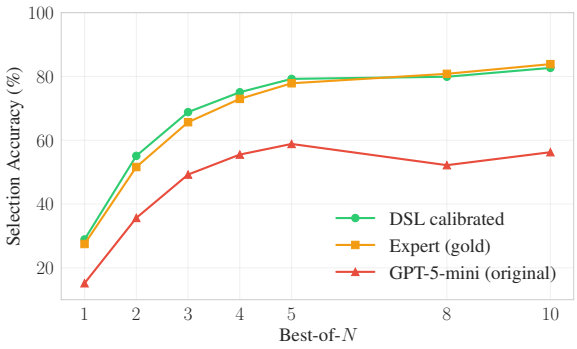


Figure 11: Best-of-N trajectory selection accuracy (7,348 problems). DSL-calibrated labels (green) improve over original GPT-5-mini labels (red) by $+13.7$pp (top-1), $+19.5$pp (top-2), and $+14.5$pp (top-4), matching or exceeding expert labels (blue).

Class-balanced-subset ablation. To isolate the effect of class imbalance, we evaluate on a class-balanced subset (84 unsound + 84 randomly sampled sound steps). Self-correction improves accuracy only 57.1%→58.3% and $F_{1,-}$ 0.333→0.327; the headline accuracy gain is largely a class-imbalance artifact, and the real benefit is calibration (matching predicted rates to true rates), which is what downstream ranking requires.

K Additional Figures and Tables

K.1 Cross-Model Results

K.2 ProcessBench Results

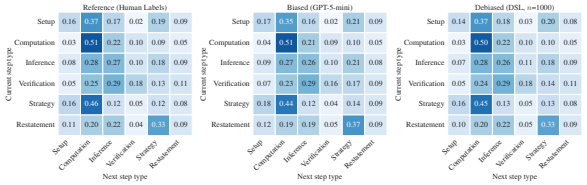


Figure 12: Transition matrices over 6 step types: (a) reference from all expert labels, (b) biased from GPT-5-mini predictions, (c) debiased via DSL at $n = 1{,}000$. The biased matrix is nearly uniform; DSL recovers the reference structure.

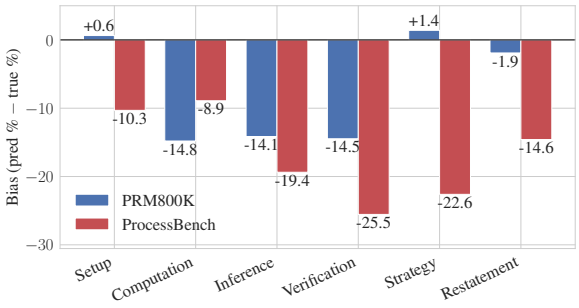


Figure 13: Per-step-type bias of GPT-5-mini on PRM800K vs ProcessBench (paired dot plot). Each line connects the same step type across datasets. Underrating is consistent across datasets but more severe on ProcessBench.

Component	Parameter	Value
GPT-5-mini ratings	Model version	`gpt-5.4-mini (version 2026-03-17)`
	Temperature / top_p	0 / 1.0
	API	Azure OpenAI
GLM-5.2 ratings	Model version	`glm-5.2`
	Temperature	0
DSL (scalar)	Cross-fitting model	GradientBoostingClassifier
	Trees / max depth	50 / 3
	Cross-fitting folds	2
	Random seed	42
DSL (transition)	Cross-fitting model	GradientBoostingClassifier
	Trees / max depth	30 / 3
	Cross-fitting folds	2
PPI	Optimizer	L-BFGS-B
	Loss	Logistic
Self-correction	Labeled set size	$n_{\text{diag}} = 1{,}000$
	Eval set size	$n_{\text{eval}} = 1{,}000$
	Rounds	1 (canonical)
	Random seed	42
Vary-expert	Repetitions	30
	n values	200, 486, 1182, 2875, …, 594689
	Random seed	42
Dataset	PRM800K	$N = 594{,}690$ steps

Table 19: Reproducibility checklist: full experimental metadata for all components.

Type	Description	Example
Setup	Define variables, set up equations	"Let x be the number of…"
Computation	Arithmetic, algebraic manipulation	"$2x + 3 = 7$, so $x = 2$"
Inference	Logical deduction from prior steps	"Since $A = 90°$, $C = 45°$"
Verification	Checking an intermediate result	"Verify: $2^2 + 2^2 = 8 \neq 9$"
Strategy	Choosing approach, planning	"We'll use induction"
Restatement	Rephrasing without new content	"Thus the answer is 42."

Table 20: Step type taxonomy for mathematical reasoning.

Type	Cost	Accuracy	Adaptable
Self-correction (1 round)	\$0.30	85.7%	Yes
Expert-only (n=1,000)	\$500	80.1%	No
Expert-only (n=1,500)	\$750	84.0%	No
Full expert annotation	\$300,000	100%	No
Fine-tuned LR/MiniLM (n=1,000)	\$50	91.6%	Retrain
Fine-tuned Qwen3.5-LoRA (n=1,000)	\$50+GPU	81.4%–86.1%	Retrain

Table 21: Cost comparison of debiasing methods. Self-correction adds \$0.30 in API calls on top of the 1,000 labels (\$500) required for diagnosis; it is 1,000× cheaper than expert-only at matched accuracy and 1,000,000× cheaper than full annotation.

N	Original	Expert	DSL	$\Delta_{\text{DSL-Expert}}$	$\Delta_{\text{DSL-Orig}}$
1	15.2%	27.5%	28.9%	+1.4pp	+13.7pp
2	35.6%	51.5%	55.1%	+3.6pp	+19.5pp
3	53.6%	68.8%	71.6%	+2.8pp	+18.0pp
4	67.2%	80.0%	81.6%	+1.6pp	+14.5pp
5	76.0%	87.0%	87.9%	+0.9pp	+11.9pp
8	88.8%	95.4%	95.2%	−0.2pp	+6.4pp
10	93.0%	97.4%	97.2%	−0.2pp	+4.2pp

Table 22: Full Best-of-N trajectory selection accuracy on 7,348 MATH problems. DSL-calibrated labels outperform Expert Gold Standard at N=1 through N=5 (where calibration matters most), while Expert slightly edges ahead at N=8 and N=10 (where most correct traces are already captured).

Run	Original Acc	Corrected Acc	Round Δ	Total Δ
Canonical (single round)	79.0%	85.7%	+6.7pp	+6.7pp
Run 2 (single round)	79.0%	85.1%	+6.1pp	+6.1pp
Run 3 (single round)	79.0%	85.5%	+6.5pp	+6.5pp
Cross-model GPT (single round)	79.0%	84.9%	+5.9pp	+5.9pp
Mean	**79.0%**	**85.3%**	+6.3pp	+6.3pp

Table 23: Single-round prompt self-correction across four independent runs (mean 85.3%, SD 0.4pp). A multi-round run on the same eval set converges at round 1 (84.3%→85.3%, +1.0pp) with round 2 adding nothing; a single round captures the full benefit.

Method	Accuracy	Δ	$F_{1,-}$	Recall$_-$	$F_{1,+}$	Balanced
Original GPT-5-mini	79.0%	—	0.146	0.214	0.880	52.9%
Self-correction	85.7%	+6.7pp	0.192	0.202	0.921	56.0%
Global threshold shift	84.6%	+5.6pp	0.083	0.083	0.916	50.0%
Platt scaling	91.6%	+12.6pp	0.00	0.00	0.956	50.0%
Always predict 1	91.6%	+12.6pp	0.00	0.00	0.956	50.0%
Fine-tuned LR	91.6%	+12.6pp	0.00	0.00	0.956	50.0%
Fine-tuned MiniLM	91.6%	+12.6pp	0.00	0.00	0.956	50.0%
Qwen3.5-0.8B (LoRA)	81.4%	+2.4pp	0.238	0.345	0.894	62.0%
Qwen3.5-2B (LoRA)	66.2%	−12.8pp	0.228	0.595	0.784	66.3%
Qwen3.5-4B (LoRA)	86.1%	+7.1pp	0.206	0.214	0.924	58.7%
Qwen3.5-9B (LoRA)	82.9%	+3.9pp	0.182	0.226	0.905	56.2%

Table 24: Baseline comparison on PRM800K ($n_{\text{eval}} = 1{,}000$, 84 unsound steps). All methods use the same n=1,000 labeled steps. $F_{1,-}$ and Recall$_-$ are for identifying *unsound* steps; accuracy-maximizing baselines (Platt, always-1, fine-tuned LR/MiniLM) detect no unsound steps. The Qwen3.5-LoRA verifiers are the only baselines that do *not* maximize accuracy: their checkpoints are selected by $F_{1,-}$ (unsound-step detection) with class-weighted loss on a held-out validation set, and all metrics are reported on the same held-out 1,000-step evaluation set. Even so, they detect only a fraction of the unsound steps ($F_{1,-}$ 0.18–0.24) and remain below the majority-class accuracy ceiling; two of the four (0.8B, 2B) exceed self-correction's $F_{1,-}$ (0.238, 0.228 vs. 0.192) but at substantially lower accuracy. Balanced accuracy = (TPR + TNR)/2 accounts for class imbalance.

Method	Naive SE	Cluster SE	Ratio
Expert-only	0.032	0.104	3.3×
PPI	0.041	0.142	3.5×
DSL	0.038	0.115	3.0×

Table 25: Naive vs. cluster-robust standard errors at $n = 1{,}000$.

Step Type	N	Original Acc	Corrected Acc	Δ
Computation	357	75.9%	86.3%	+10.4pp
Verification	67	73.1%	82.1%	+9.0pp
Restatement	90	82.2%	87.8%	+5.6pp
Setup	119	84.9%	89.9%	+5.0pp
Inference	176	77.8%	82.4%	+4.5pp
Strategy	191	82.7%	85.3%	+2.6pp

Table 26: Per-step-type accuracy after single-round correction. Most under-rated types see the largest gains.

Feedback	Acc	Δ	Share
No feedback (baseline)	79.0%	—	—
Overall only	85.9%	+6.9pp	130%
Type only	85.2%	+6.2pp	117%
Position only	80.9%	+1.9pp	36%
Full feedback	84.3%	+5.3pp	100%
Canonical full feedback[†]	85.7%	+6.7pp	126%

Table 27: Ablation: accuracy improvement by feedback component. All variants use the same 1,000-step eval set. [†]Canonical single-round run (headline result); differs from the pipeline's round-0 full feedback due to run-to-run variance in API calls.

Setting	$F_{1,-}$ Orig→Corr	Δ_{F1}	Cal Gap Orig→Corr	Δ_{Cal}
PRM800K (GPT-5-mini)	0.146 → 0.192	+32%	7.8pp → 0.9pp	×8.7
PRM800K (GLM-5.2†)	0.143 → 0.138	−3%	1.4pp → 2.3pp	+0.9pp
ProcessBench	0.523 → 0.556	+6%	13.0pp → 4.2pp	×3.1

Table 28: Calibration and error-detection metrics. Self-correction primarily improves calibration (matching predicted rates to true rates) and $F_{1,-}$ on unsound steps. †GLM-5.2 uses feedback generated from GPT-5-mini's bias profile; cross-model transfer improves accuracy but not error detection.

Step	Type	GPT	After	Human
6	strategy	✓	✓	✓
7	setup	✓	✓	✓
8	**computation**	×	✓	✓
9	computation	✓	✓	✓
15	computation	✓	✓	✓

Table 29: Qualitative example: "*Given* $f(x) = x^2 - 2x + 5$ *and* $g(x) = x + 3$, *find* $f(g(5)) - g(f(5))$." Step 8 ("$g(f(x)) = f(x) + 3$") is a computation step at position 0.44 that GPT-5-mini incorrectly rates as unsound (×); feedback flips it to sound (✓), matching the human label.

	RMSE (mean ± SE)		DSL vs Expert		DSL vs PPI	
n	DSL	Expert-only	t-test p	Wilcoxon p	t-test p	Wilcoxon p
200	0.84 ± 0.08	6.09 ± 0.99	$< 0.001^{***}$	$< 0.001^{***}$	$< 0.001^{***}$	$< 0.001^{***}$
486	0.45 ± 0.03	0.83 ± 0.24	0.83	0.002^{**}	$< 0.001^{***}$	$< 0.001^{***}$
1,182	0.29 ± 0.03	0.31 ± 0.02	> 0.99	> 0.99	0.30	$< 0.001^{***}$
2,875	0.33 ± 0.02	0.24 ± 0.02	$< 0.001^{***}$	$< 0.001^{***}$	0.55	> 0.99

Table 30: Statistical comparison of debiasing methods (30 repetitions, paired tests, Bonferroni-corrected for 8 comparisons). $^{***}p < 0.001$, $^{**}p < 0.01$. DSL significantly outperforms both baselines at $n = 200$; Expert-only significantly outperforms DSL at $n \geq 2{,}875$ (crossing phenomenon). DSL significantly outperforms PPI at small n (200, 486) but not at moderate n ($p = 0.30$ at $n = 1{,}182$, $p = 0.55$ at $n = 2{,}875$).

Method	Description	Code
Reference	All-N expert labels	IRLS
Expert-only	n expert labels only	IRLS
Imputation	All-N predicted labels	IRLS
PPI	Gradient rectification	L-BFGS-B
DSL	Honest forest + pseudo-outcome	IRLS

Table 31: Methods compared in all experiments.

Bias Dimension	GPT-5-mini	GLM-5.2
Overall (% pred − % true)	−7.8pp	−1.4pp
Computation bias	−14.6pp	−2.0pp
Verification bias	−11.9pp	−6.0pp
Position correlation	−0.25	−0.12
Length correlation	−0.03	−0.08

Table 32: Bias profile comparison across models, computed on the shared 1,000-step evaluation set for both models.

Model	Original	Corrected	Δ
GPT-5-mini	79.0%	84.9%	+5.9pp
GLM-5.2	84.4%	87.5%	+3.1pp

Table 33: Self-correction gain across model families after a single correction round, using identical DSL-diagnosed feedback. Both models benefit from the same feedback. The GPT-5-mini result here (84.9%, +5.9pp) is from the cross-model transfer experiment with a different random seed than the canonical headline run (85.7%, +6.7pp); see Section 5.2 for discussion of run-to-run variance.

Split	N steps	Original	Corrected	Δ
GSM8K	1,568	94.6%	92.4%	−2.2pp
MATH	4,366	85.9%	89.7%	+3.8pp
OlympiadBench	5,822	82.0%	85.7%	+3.7pp
OmniMath	4,792	72.3%	83.0%	+10.7pp
Total	16,548	81.4%	86.6%	+5.2pp

Table 34: Self-correction on ProcessBench. Improvement on 3 of 4 splits; gains scale with initial bias severity.

Method	Accuracy	95% CI
Zero-shot	82.8%	[80.3, 85.0]
Few-shot (k=1)	83.7%	[81.3, 85.9]
Few-shot (k=3)	83.7%	[81.3, 85.9]
Few-shot (k=5)	83.6%	[81.1, 85.8]
Few-shot (k=10)	84.0%	[81.6, 86.1]
DSL + feedback (self-correction)	87.9%	[85.7, 89.8]
DSL + feedback + few-shot (k=5)	87.8%	[85.6, 89.7]

Table 35: Few-shot baselines on ProcessBench. Self-correction outperforms few-shot prompting.